

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO
CAMPUS SÃO PAULO**

TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

**JOAO ANTÔNIO CHAMUSCA MARTINS
KELVYN GABRIEL MILU DE OLIVEIRA
LUCAS VIEIRA DE SOUZA
MATHEUS PEREIRA PEREIRA**

**SISTEMA INTELIGENTE DE MONITORAMENTO E SEGURANÇA
CIBERNÉTICA: APOIO À PROTEÇÃO E ORIENTAÇÃO SOBRE A
VULNERABILIDADE DIGITAL (SafeID)**

SÃO PAULO

2026

JOAO ANTÔNIO CHAMUSCA MARTINS
KELVYN GABRIEL MILU DE OLIVEIRA
LUCAS VIEIRA DE SOUZA
MATHEUS PEREIRA PEREIRA

SISTEMA INTELIGENTE DE MONITORAMENTO E SEGURANÇA
CIBERNÉTICA: APOIO À PROTEÇÃO E ORIENTAÇÃO SOBRE A
VULNERABILIDADE DIGITAL (SafeID)

Projeto apresentado ao Programa do Curso de Tecnologia em Análise e Desenvolvimento de Sistemas, do Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, Campus de São Paulo, como requisito parcial para aprovação na disciplina SPOPIE1 – Projeto Integrado de Extensão 1, sob orientação do Professor Anderson Gomes Peixoto.

SÃO PAULO

2026

RESUMO

A crescente dependência de serviços digitais por indivíduos e organizações tem gerado um rastro massivo de informações pessoais e de acesso restrito, tornando-as vulneráveis a incidentes de segurança. No cenário atual, a exfiltração de dados em larga escala e a subsequente compilação de bases de credenciais expostas em repositórios de *threat intelligence* representam um risco crítico à privacidade e à integridade dos titulares. Diante dessa realidade, este trabalho apresenta o SafeID, um sistema desenvolvido para permitir ao usuário (cadastrado ou não) monitorar exposições registradas e oferecer apoio e orientação sobre sua vulnerabilidade digital.

O objetivo central do projeto é consolidar uma plataforma que atue no processamento e na triagem de informações provenientes de fontes externas que catalogam esses incidentes de segurança. Ao contrário de ferramentas de detecção em tempo real, o SafeID foca no mapeamento de dados que já foram comprometidos e estão circulando em repositórios de ameaças conhecidos. O sistema busca democratizar o acesso a essa informação técnica sobre o contexto da vulnerabilidade digital, permitindo que o cidadão comum compreenda se seus dados cadastrais ou credenciais de acesso fazem parte de vazamentos documentados, facilitando a gestão de sua própria identidade digital.

Metodologicamente, a pesquisa utiliza uma abordagem de desenvolvimento ágil, com ciclos iterativos de análise, implementação e validação. A solução é composta por uma interface desenvolvida em React e por serviços de *back-end* em Node.js com NestJS, com consulta a fontes de *threat intelligence* e cálculo de um *score* ponderado na PI-1; para a PI-2, está prevista a integração com modelos de linguagem de grande escala (LLMs) para converter metadados técnicos em orientações acessíveis. O diagnóstico é apoiado por um *score* ponderado segundo a criticidade dos dados expostos. A validação funcional e os resultados de desempenho (latência e vazão) foram obtidos com dados sintéticos em ambiente simulado e têm caráter preliminar, sem tratamento desnecessário de dados pessoais reais. Ao final, o projeto indica que a combinação dessas tecnologias pode apoiar a prevenção e a comunicação de riscos, desde que acompanhada de governança, controles técnicos e observância das diretrizes da LGPD.

Palavras-chave: Segurança cibernética. Inteligência de Ameaças. LGPD. Inteligência Artificial. SafeID.

ABSTRACT

The growing reliance on digital services by individuals and organizations has generated a massive trail of personal and restricted access information, making them vulnerable to security incidents. In the current scenario, large-scale data exfiltration and the subsequent compilation of exposed credential databases in threat intelligence repositories represent a critical risk to the privacy and integrity of data subjects. Given this reality, this paper presents SafeID, a system developed to allow users (registered or not) to monitor recorded exposures and offer support and guidance on their digital vulnerability.

The central objective of the project is to consolidate a platform that operates in the processing and screening of information from external sources that catalog these security incidents. Unlike real-time detection tools, SafeID focuses on mapping data that has already been compromised and is circulating in known threat repositories. The system seeks to democratize access to this technical information regarding the context of digital vulnerability, allowing ordinary citizens to understand if their registration data or access credentials are part of documented leaks, facilitating the management of their own digital identity.

Methodologically, the research uses an agile development approach to create an application based on React and Node.js with NestJS as back-end framework. The system's differential lies in the combination of a weighted risk score, implemented in PI-1, with artificial intelligence models (LLMs), planned for PI-2, to translate technical incident records into practical mitigation guides. Furthermore, SafeID proposes as a value delivery a visual risk index — the "exposure score" — and, in PI-2, personalized action plans. The latency and throughput performance results presented were obtained using synthetic validation data in a simulated environment and should be regarded as preliminary. In this way, the platform is designed in alignment with the guidelines of the General Data Protection Law (LGPD) by strengthening informative self-determination and the culture of preventive security in Brazil.

Keywords: Cybersecurity. Threat Intelligence. LGPD. Artificial Intelligence. SafeID.

ÍNDICE DE FIGURAS

Figura 1 - Board de Atividades do SafeID no GitHub Projects	23
Figura 2 - Diagrama de arquitetura do SafeID	47
Figura 3 - Modelo Entidade Relacionamento (MER) SafeID	67
Figura 4 - Diagrama entidade-relacionamento SafeID	68
Figura 5 - Roadmap de desenvolvimento de atividades do SafeID	73

ÍNDICE DE QUADROS

Quadro 1 - Comparativo de Funcionalidades entre Concorrentes	13
Quadro 2 - Matriz de Responsabilidades e Atribuições da Equipe	21
Quadro 3 - Matriz de Riscos do Projeto SafeID.....	23
Quadro 4 - Backlog do Produto (Product Backlog - PI-1 e PI-2)	27
Quadro 5 - Requisitos Funcionais (RF)	33
Quadro 6 - Requisitos Não Funcionais (RNF)	34
Quadro 7 - Matriz de Rastreabilidade (Requisitos vs. Testes)	59
Quadro 8 - Dicionário de dados do SafeID	69
Quadro 9 - Cronograma Consolidado e Macro-Fases de Engenharia do Projeto.....	72

ÍNDICE DE TABELAS

Tabela 1 - Categorias e pesos dos metadados obtidos da API.....	38
Tabela 2 - Validação de Desempenho e Carga (K6 e Resiliência).....	57
Tabela 3 - Matriz de Pesos e Criticidade do Score de Risco (RN02 e RN03).....	58
Tabela 4 - Cobertura da suíte unitária do back-end (medição de 06/10/2026).....	62
Tabela 5 - Custos operacionais mensais (OpEx) estimados por categoria	74

ÍNDICE DE ABREVIACÕES E SIGLAS

ACID – Atomicidade, Consistência, Isolamento e Durabilidade
AES – Advanced Encryption Standard (Padrão de Criptografia Avançada)
API – Application Programming Interface (Interface de Programação de Aplicações)
AWS – Amazon Web Services
B2C – Business-to-Consumer
B2B2C – Business-to-Business-to-Consumer
CI/CD – Continuous Integration / Continuous Deployment (Integração Contínua / Implantação Contínua)
CNPJ – Cadastro Nacional da Pessoa Jurídica
CPF – Cadastro de Pessoas Físicas
CPU – Central Processing Unit (Unidade Central de Processamento)
CRUD – Create, Read, Update, Delete (Criar, Ler, Atualizar, Deletar)
CSS – Cascading Style Sheets (Folhas de Estilo em Cascata)
DBA – Database Administrator (Administrador de Banco de Dados)
DDL – Data Definition Language (Linguagem de Definição de Dados)
DER – Diagrama Entidade-Relacionamento
DML – Data Manipulation Language (Linguagem de Manipulação de Dados)
ECS – Elastic Container Service
GCM – Galois/Counter Mode
HIBP – Have I Been Pwned
IA – Inteligência Artificial
IFSP – Instituto Federal de Educação, Ciência e Tecnologia de São Paulo
JSON – JavaScript Object Notation (Notação de Objetos JavaScript)
JWT – JSON Web Token
LGPD – Lei Geral de Proteção de Dados Pessoais
LLM – Large Language Model (Modelo de Linguagem de Grande Escala)
MFA – Multi-Factor Authentication (Autenticação em Múltiplos Fatores)
MVP – Minimum Viable Product (Produto Mínimo Viável)
NLP – Natural Language Processing (Processamento de Linguagem Natural)
ORM – Object-Relational Mapping (Mapeamento Objeto-Relacional)
OSINT – Open Source Intelligence (Inteligência de Fontes Abertas)
PII – Personally Identifiable Information (Informações Pessoais Identificáveis)
PO – Product Owner (Dono do Produto)
PoC – Proof of Concept (Prova de Conceito)
RAM – Random Access Memory (Memória de Acesso Aleatório)
RDS – Relational Database Service
SBDG – Sistema de Gerenciamento de Banco de Dados
SHA – Secure Hash Algorithm
UDF – User Defined Functions (Funções Definidas pelo Usuário)
UML – Unified Modeling Language (Linguagem de Modelagem Unificada)
UX – User Experience (Experiência do Usuário)
VPC – Virtual Private Cloud

SUMÁRIO

1. INTRODUÇÃO.....	9
1.1 Objetivos.....	9
1.1.1 Objetivo Geral	9
1.1.2 Objetivos Específicos	10
1.2. Problema e Solução Proposta	10
1.2.1 Problema.....	10
1.2.2 Solução Proposta	11
1.3 Justificativa.....	11
1.4 Análise da Concorrência.....	12
1.4.1 Have I Been Pwned (HIBP).....	12
1.4.2 Mozilla Monitor.....	12
1.4.3 Soluções de Identidade (Ex: LifeLock, Aura)	12
1.5 Quadro Comparativo de Funcionalidades	13
1.6 Diferencial da Proposta e Organização do Trabalho	13
2 REVISÃO DA LITERATURA	15
2.1 A Economia da Informação e a Evolução das Ameaças Digitais.....	15
2.2 Custódia de Dados, Fadiga de Segurança e o "Gap" de Detecção	16
2.3 <i>Risk Scoring</i> e Modelos de Pontuação de Risco em Segurança	16
2.4 Marco Regulatório (LGPD) e a Literacia Digital no Brasil	17
2.5 Síntese e Lacuna de Pesquisa	18
3 GESTÃO DO PROJETO	20
3.1 Organização da Equipe.....	20
3.1.1 Responsabilidades e Atividades	20
3.2 Metodologias de Gestão e Desenvolvimento	22
3.2.1 Scrum (Metodologia Ágil).....	22
3.2.2 Gestão de Riscos do Projeto	23
3.3 Repositório da Aplicação.....	26
3.3.1 Definição do Repositório.....	26
3.3.2 Link do Repositório e Especificações para Acesso	26
3.3.3 Backlog do Produto (Product Backlog).....	27
3.3.4 Detalhamento das Sprints (Projeto Integrado I - Março a Junho)	29
3.3.5 Planejamento para Projeto Integrado II (Agosto a Dezembro)	29
4 DESENVOLVIMENTO DO PROJETO.....	30
4.1 Escopo do projeto	30
4.1.1 Regras de negócio.....	31
4.1.2 Requisitos funcionais.....	32
4.1.3 Requisitos não funcionais.....	34
4.1.4 Modelo de Avaliação de Risco e Ponderação de Exposição (Risk Score).....	36
4.2 Histórias de usuário	42
4.2.1 Descrição das Histórias de Usuário	42
4.3 Arquitetura.....	45
4.3.1 Definições da arquitetura.....	45
4.3.2 Diagrama da arquitetura	47
4.4 Tecnologias.....	48
4.4.1 Front-end: React.js e Tailwind CSS	48
4.4.2 Back-end: Node.js, NestJS e Prisma ORM	49

4.4.3 Banco de Dados: PostgreSQL	50
4.4.4 Inteligência Artificial e APIs Externas	51
4.4.5 Infraestrutura em Cloud	52
4.5 Testes e Manutenibilidade	53
4.5.1 Plano de Testes	54
4.5.2 Análise Estática	54
4.5.3 Testes Funcionais	54
4.5.4 Testes Não Funcionais	56
4.5.5 Testes Automatizados	60
4.5.6 Logs	63
4.5.7 Code Convention	63
4.6 Segurança, Privacidade e Legislação	64
4.6.1 Critérios de Segurança e Privacidade	64
4.6.2 Observância à Legislação	65
4.7 Modelo de Banco de Dados	66
4.7.1 Modelo Entidade-Relacionamento (MER)	66
4.7.2 Diagrama Entidade-Relacionamento (DER)	67
4.7.3 Dicionário de Dados	68
4.8 Duração / Cronograma	70
4.8.1 Análise da Duração do Projeto	71
5 VIABILIDADE FINANCEIRA	74
5.1 Custos	74
5.2 Receitas	75
5.3 Cenário Realista	76
5.4 Cenário Otimista	76
5.5 Cenário Pessimista	77
5.6 Métricas SaaS: CAC e LTV	78
6 CONSIDERAÇÕES FINAIS	79
6.1 Verificação dos Objetivos Específicos	79
6.2 Dificuldades	79
6.2.1 Dificuldades Técnicas Encontradas	79
6.3 Escolhas e Descartes	80
6.3.1 Escolhas Arquiteturais Consolidadas	80
6.3.2 Descartes Estratégicos	81
6.4 Limitações	81
6.5 Trabalhos Futuros	82
REFERÊNCIAS BIBLIOGRÁFICAS	83

1. INTRODUÇÃO

A ampliação da circulação de dados pessoais em serviços digitais elevou a exposição dos titulares a incidentes cibernéticos. Em 2022, o Brasil registrou mais de 30 bilhões de tentativas de ataques cibernéticos, o segundo maior volume da América Latina (FORTINET, 2022). Esse indicador dimensiona a superfície de ataque, mas não informa ao titular quais de seus dados foram comprometidos, qual é a gravidade da exposição ou como reagir.

Essa lacuna caracteriza uma assimetria informacional: enquanto especialistas e agentes maliciosos operam com registros técnicos e bases de credenciais expostas, o usuário comum recebe alertas fragmentados, linguagem especializada e recomendações genéricas. Mesmo quando toma conhecimento de um vazamento, pode não compreender o risco nem priorizar medidas de mitigação (CONTI; DARGAHI; DEGHANTANHA, 2018a; CHEN; WANG, 2024).

O problema do SafeID, portanto, não se limita à identificação de vazamentos. Ele envolve transformar evidências técnicas em comunicação de risco compreensível e acionável, considerando barreiras como a reutilização de senhas (CONTI et al., 2018b), a baixa adoção de autenticação em múltiplos fatores e o desconhecimento sobre exposições documentadas (NEMEC ZLATOLAS; WELZER; LHOTSKA, 2024). Sem essa mediação, a informação disponível tende a produzir inércia, e não prevenção.

Para enfrentar essa assimetria, este trabalho propõe o SafeID, plataforma web cuja arquitetura prevê a consulta de exposições registradas em fontes de inteligência de ameaças, o cálculo de um índice de risco e a tradução dos resultados em orientações acessíveis. Neste documento, a PI-1 designa a etapa já realizada (março a junho de 2026), na qual foram implementadas a consulta às exposições e a avaliação determinística do risco; a PI-2 designa a etapa planejada (agosto a dezembro de 2026), que prevê a integração de inteligência artificial para comunicar a gravidade de cada incidente e indicar ações de mitigação compatíveis com os dados comprometidos (REUBEN-OWOH; HAIG, 2025).

1.1 Objetivos

Esta seção apresenta os objetivos que orientam o desenvolvimento do SafeID. Estabelece-se primeiro o objetivo geral e, em seguida, os objetivos específicos, cada um identificado pela fase do projeto (PI-1 ou PI-2) para permitir a verificação de seu status na Seção 6.1.

1.1.1 Objetivo Geral

Desenvolver a plataforma SafeID, um sistema de apoio à decisão voltado à segurança cibernética preventiva, capaz de processar informações de repositórios de *threat intelligence*

sobre incidentes de exposição de dados, diagnosticar a exposição de dados pessoais do usuário e fornecer diretrizes de mitigação personalizadas por meio de inteligência artificial.

1.1.2 Objetivos Específicos

Os objetivos específicos a seguir detalham as etapas necessárias para o cumprimento do objetivo geral e indicam a fase do projeto (PI-1 ou PI-2) em que cada um é tratado:

- **Investigar, selecionar e consumir** APIs de monitoramento de exposições de incidentes de segurança *Open-Source Intelligence* (OSINT) que permitam a triagem automatizada de credenciais e dados cadastrais expostos (Fase PI-1, concluído);
- **Modelar e implementar** um algoritmo de cálculo de *Risk Score* (índice de risco) que pondere a criticidade do vazamento com base na natureza dos dados comprometidos reportados pelas APIs (Fase PI-1, concluído);
- **Desenvolver** uma interface web responsiva que utilize elementos de visualização de dados (como o termômetro de risco) para facilitar a interpretação dos usuários (Fase PI-1, concluído);
- Projetar a arquitetura de persistência para o histórico de consultas e os metadados dos incidentes (base implementada na PI-1; funções de conta e histórico completo previstas para a PI-2);
- **Integrar** um modelo de linguagem de grande escala (LLM) para atuar como assistente cognitivo, traduzindo metadados técnicos em planos de ação práticos e acessíveis (Fase PI-2, planejado);
- **Aplicar** os princípios de *Privacy by Design* e as diretrizes da Lei Geral de Proteção de Dados (LGPD), assegurando o tratamento ético e a proteção das informações coletadas durante o ciclo de uso do sistema (Fase PI-2, planejado).

1.2. Problema e Solução Proposta

Esta seção delimita o problema que fundamenta o SafeID e apresenta a solução concebida para enfrentá-lo.

1.2.1 Problema

O problema tratado pelo SafeID é a distância entre a existência de informações sobre vazamentos e a capacidade do titular de interpretá-las. Bases de credenciais exfiltradas circulam em repositórios de *threat intelligence* e em mercados secundários, e o custo médio global de um vazamento de dados atingiu US\$ 4,45 milhões em 2023, 15% acima do valor registrado três anos antes (IBM SECURITY, 2023). Do lado do titular, contudo, as ferramentas de consulta

existentes frequentemente entregam resultados em formato técnico bruto, que não permite interpretação imediata do risco (CONTI; DARGAHI; DEHGHANTANHA, 2018a).

Essa lacuna gera inércia: mesmo ciente do vazamento, o indivíduo não compreende a gravidade da exposição nem quais contramedidas priorizar. A questão que orienta este trabalho é, portanto, como converter registros técnicos de exposição em uma medida de risco compreensível e em orientações acionáveis para o usuário comum.

1.2.2 Solução Proposta

Diante dessa lacuna, o SafeID estrutura-se como uma plataforma de apoio à decisão voltada à fase pós-incidente. Na PI-1, foram implementados o processamento de metadados provenientes de APIs de monitoramento de exposições, como o Have I Been Pwned (HIBP), e os componentes de consulta e de cálculo do risco. A PI-2 prevê a consolidação dos recursos de persistência, autenticação, monitoramento ativo e assistência cognitiva. A solução organiza-se em três componentes centrais:

1. **Mapeamento Automatizado (implementado na PI-1):** realiza a triagem de incidentes vinculados ao identificador do usuário em repositórios de ameaças;
2. **Cálculo de *Risk Score* (implementado na PI-1):** aplica um algoritmo ponderado que quantifica a gravidade da exposição com base na natureza do dado, atribuindo, por exemplo, peso superior às senhas em relação aos nomes;
3. **Assistente Cognitivo (trabalho futuro da PI-2):** utilizará modelos de linguagem (LLMs) para converter diagnósticos técnicos em planos de ação personalizados e compreensíveis.

O protótipo da PI-1 permite a consulta e a visualização do risco. Na PI-2, a incorporação do assistente cognitivo e dos demais recursos planejados deverá ampliar a capacidade de orientar o usuário na adoção de contramedidas.

1.3 Justificativa

A relevância do projeto está na necessidade de reduzir as barreiras técnicas que dificultam ao cidadão exercer sua autodeterminação informativa, princípio assegurado pela Lei Geral de Proteção de Dados (BRASIL, 2018). A literatura aponta que práticas inadequadas dos próprios titulares, como a reutilização de senhas e a baixa adoção de autenticação em múltiplos fatores, ampliam o risco decorrente de vazamentos (NEMEC ZLATOLAS; WELZER; LHOTSKA, 2024).

A parceria com a Enetsec aproxima o desenvolvimento acadêmico da experiência do setor privado. A empresa contribui com consultoria especializada e com a validação da proposta

por profissionais de segurança da informação, o que apoia a verificação dos requisitos do projeto e a análise de viabilidade técnica. Essa colaboração também sustenta a evolução prevista para um modelo B2B2C, no qual organizações parceiras disponibilizariam a seus clientes recursos de monitoramento, avaliação de risco e orientação preventiva.

A justificativa organiza-se em três dimensões:

- **Social:** a ferramenta oferece orientação clara em um cenário em que o desconhecimento sobre vazamentos facilita fraudes financeiras e o roubo de identidade;
- **Tecnológica:** o projeto combina a consulta a APIs de segurança e, na PI-2, o processamento de linguagem natural para tratar um problema de comunicação de risco, em linha com a defesa de mensagens personalizadas feita por Reuben-Owoh e Haig (2025);
- **Normativa:** ao facilitar o diagnóstico de exposições históricas, o sistema apoia o direito de acesso à informação, pois a mera divulgação de fatores de risco, sem estrutura interpretativa, não informa adequadamente o titular (CHEN; HENRY; JIANG, 2023).

1.4 Análise da Concorrência

O mercado oferece desde agregadores de incidentes até suítes de proteção de identidade. A análise a seguir, baseada nos critérios descritos na Seção 1.5, indica que a maioria das soluções entrega o dado bruto, sem uma camada de interpretação voltada ao usuário leigo.

1.4.1 *Have I Been Pwned (HIBP)*

O HIBP é um agregador de incidentes de referência e a principal fonte de dados do SafeID (HAVE I BEEN PWNEED, [s. d.]). Sua interface é técnica e informativa.

- **Limitação:** o serviço opera no modelo "consulta e resposta", sem métrica de gravidade (*score*) nem plano de mitigação além de sugestões genéricas, como a troca de senhas.

1.4.2 *Mozilla Monitor*

O serviço da Mozilla atua como camada de interface sobre os dados do HIBP, com experiência mais simples para o usuário.

- **Limitação:** embora ofereça alertas por e-mail, as recomendações permanecem estáticas, sem análise cognitiva dos vazamentos para gerar orientações específicas a cada perfil de incidente.

1.4.3 *Soluções de Identidade (Ex: LifeLock, Aura)*

São plataformas comerciais pagas que oferecem monitoramento extensivo, incluindo pontuação de crédito e vigilância contínua.

- **Limitação:** além do custo, apresentam configuração mais complexa e costumam estar integradas a serviços bancários e de crédito, o que as distancia de uma ferramenta ágil e focada na literacia em cibersegurança do cidadão comum.

1.5 Quadro Comparativo de Funcionalidades

A classificação dos concorrentes foi realizada por meio de análise comparativa da documentação oficial de cada solução e de testes manuais das funcionalidades disponíveis em suas versões gratuitas, quando aplicável. Foram observados critérios comuns de consulta a incidentes, alertas, acessibilidade da interface, cálculo de risco, uso de assistência cognitiva e foco em educação digital. Por tratar-se de verificação qualitativa e sujeita à evolução das ferramentas, o Quadro 1 deve ser lida como retrato do momento da análise.

Quadro 1 - Comparativo de Funcionalidades entre Concorrentes

Funcionalidade	HIBP	Mozilla Monitor	Soluções Pagas	SafeID
Consulta de incidentes (OSINT)	Sim	Sim	Sim	Sim
Alertas de novas exposições	Não*	Sim	Sim	Sim (PI-2)
Interface Acessível	Não	Sim	Parcial	Sim
Score de Risco Ponderado	Não	Não	Sim	Sim
Assistente Cognitivo (LLM)	Não	Não	Não	Sim (PI-2)
Foco em Educação Digital	Não	Parcial	Não	Sim

*A funcionalidade gratuita de consulta do HIBP não inclui alertas ativos para o usuário comum sem registro de domínio.

1.6 Diferencial da Proposta e Organização do Trabalho

O diferencial do SafeID está na combinação de uma experiência centrada no usuário, da apresentação acessível do nível de risco e da tradução de dados técnicos em orientações práticas. Layout, arquitetura, componentes e diagramas são detalhados exclusivamente no Capítulo 4.

O trabalho organiza-se em seis capítulos. Após esta introdução, o Capítulo 2 apresenta a revisão da literatura e a lacuna de pesquisa; o Capítulo 3 descreve a gestão do projeto; o Capítulo 4 trata do desenvolvimento (escopo, requisitos, arquitetura, tecnologias, testes, segurança, modelo de dados e cronograma); o Capítulo 5 analisa a viabilidade financeira; e o Capítulo 6

reúne as considerações finais, com a verificação dos objetivos, as limitações e os trabalhos futuros.

2 REVISÃO DA LITERATURA

Este capítulo reúne os fundamentos teóricos e normativos que sustentam o SafeID. A revisão percorre a evolução das ameaças na economia da informação (2.1), os desafios de custódia e comunicação de incidentes (2.2), os modelos de pontuação de risco (2.3) e a relação entre a LGPD e a literacia digital no Brasil (2.4). O capítulo se encerra com uma síntese que declara a lacuna de pesquisa (2.5).

2.1 A Economia da Informação e a Evolução das Ameaças Digitais

A economia digital elevou os dados pessoais à condição de ativos estratégicos, distribuídos entre serviços, dispositivos e organizações. Von Solms e Van Niekerk (2013) interpretam essa mudança como uma ampliação do próprio campo da cibersegurança: a proteção deixa de se limitar à infraestrutura corporativa e passa a abranger a identidade e a conduta do indivíduo. Essa perspectiva fornece o fundamento de governança pessoal do problema, mas, isoladamente, não explica como os dados comprometidos continuam produzindo risco após a violação.

Conti, Dargahi e Dehghantanha (2018a) deslocam a análise para o ciclo posterior ao comprometimento: dados exfiltrados deixam de constituir episódios isolados e passam a alimentar mercados secundários e novas campanhas abusivas. Nesse ponto, sua abordagem de *Threat Intelligence* complementa Von Solms e Van Niekerk (2013), pois transforma a responsabilidade ampla de proteção em atividade baseada em evidências. Há, contudo, uma tensão relevante: enquanto a inteligência de ameaças tende a privilegiar capacidade analítica e antecipação técnica, a governança pessoal exige que seus resultados sejam compreensíveis e acionáveis pelo titular. A efetividade da defesa depende, portanto, tanto da qualidade da evidência coletada quanto de sua tradução para decisões individuais.

No Brasil, o volume de tentativas de ataque registrado em 2022 (FORTINET, 2022), já apresentado na Introdução, dimensiona a superfície de ataque, mas não permite, por si só, avaliar a gravidade de cada exposição. À luz de Von Solms e Van Niekerk (2013), ele evidencia a necessidade de proteção orientada ao indivíduo; sob a perspectiva de Conti, Dargahi e Dehghantanha (2018a), demonstra a importância de organizar evidências dispersas para identificar padrões e priorizar respostas. Essa convergência sustenta ferramentas que não apenas localizem incidentes, mas também convertam seu conteúdo em informação útil ao usuário.

2.2 Custódia de Dados, Fadiga de Segurança e o "Gap" de Detecção

A proteção individual é condicionada pela forma como organizações custodiam, detectam e comunicam incidentes envolvendo dados pessoais. A IBM Security (2023) estima em aproximadamente 277 dias o ciclo médio entre a ocorrência e a contenção de um vazamento. Esse intervalo mostra que a governança defendida por Von Solms e Van Niekerk (2013) não pode ser transferida integralmente ao indivíduo: durante meses, o titular pode desconhecer uma exposição originada em sistemas que não controla. A inteligência baseada em evidências proposta por Conti, Dargahi e Dehghantanha (2018a) reduz parte dessa assimetria ao correlacionar registros de ameaças, mas sua contribuição permanece incompleta se o resultado chegar tarde ou em linguagem inacessível.

Nesse intervalo entre exposição, detecção e reação, a “fadiga de segurança” amplia a vulnerabilidade. Nemec Zlatolas, Welzer e Lhotska (2024) associam a gestão complexa de credenciais e a baixa adoção de autenticação em múltiplos fatores à dificuldade de incorporar práticas protetivas à rotina. A contribuição desses autores qualifica o diagnóstico de Conti, Dargahi e Dehghantanha (2018a): dispor de inteligência sobre ameaças não garante, por si só, mudança de comportamento. Se a evidência técnica aumenta a carga cognitiva, ela pode produzir inércia em vez de prevenção. Em contraste, a perspectiva centrada no indivíduo de Von Solms e Van Niekerk (2013) exige que controles e mensagens sejam compatíveis com a capacidade real de compreensão e ação do usuário.

Chen, Henry e Jiang (2023) aprofundam esse argumento ao observar que a divulgação de fatores de risco, desacompanhada de uma estrutura interpretativa, não informa adequadamente o titular sobre o impacto da exposição. A posição converge com Nemec Zlatolas, Welzer e Lhotska (2024) quanto ao limite das respostas que apenas impõem tarefas ao usuário, porém desloca o foco da gestão de credenciais para a qualidade da comunicação. Santos e Pinto (2025) reforçam que barreiras cognitivas e linguísticas distorcem a percepção de risco, enquanto Chen e Wang (2024) defendem a integração entre resposta técnica e suporte educacional. Em conjunto, esses trabalhos indicam que reduzir o hiato de detecção requer uma cadeia completa: obtenção de evidências, avaliação de gravidade, comunicação inteligível e indicação de contramedidas. O SafeID se posiciona nessa interface entre inteligência técnica e apoio efetivo à decisão.

2.3 *Risk Scoring* e Modelos de Pontuação de Risco em Segurança

A pontuação de risco constitui um mecanismo de síntese: converte múltiplos atributos de um incidente em uma medida ordinal ou numérica destinada a apoiar priorização. Em segurança cibernética, essa quantificação não equivale a uma previsão exata do dano; trata-se

de uma representação comparável, construída a partir de dimensões como criticidade do dado, possibilidade de exploração, extensão da exposição e temporalidade. A abordagem de inteligência orientada por evidências de Conti, Dargahi e Dehghantanha (2018a) oferece a base para selecionar atributos observáveis, enquanto a concepção ampliada de cibersegurança de Von Solms e Van Niekerk (2013) impede que a métrica seja reduzida à severidade técnica: o impacto sobre identidade, privacidade e capacidade de reação do titular também deve compor a avaliação.

Modelos de referência, como o Common Vulnerability Scoring System (FIRST, 2019), ilustram a lógica de ponderar atributos e normalizar o resultado em uma escala fixa, de 0 a 10. Entretanto, esse tipo de modelo destina-se a medir a severidade técnica de vulnerabilidades, e não a exposição de dados pessoais de um titular, o que justifica um índice próprio para o problema do SafeID. De modo geral, esses modelos atribuem pesos diferenciados aos fatores de risco e normalizam o resultado em faixas interpretáveis. Essa lógica permite distinguir, por exemplo, a exposição de credenciais de autenticação de metadados menos diretamente exploráveis, bem como considerar que a recência pode alterar a probabilidade de uso abusivo. Nemec Zlatolas, Welzer e Lhotska (2024) oferecem suporte comportamental para essa diferenciação ao relacionar reutilização de credenciais, complexidade de gestão e baixa adoção de MFA; Chen, Henry e Jiang (2023), por sua vez, alertam que a pontuação só cumpre função informativa se seus critérios e consequências forem comunicados de modo compreensível. Assim, pesos e limiares devem ser explícitos, justificáveis e auditáveis, para que a aparência de precisão matemática não oculte escolhas normativas do modelo.

Para o SafeID, essa literatura fundamenta um índice explicável de exposição, no qual metadados heterogêneos são categorizados, ponderados e apresentados em níveis de prioridade. O modelo deve funcionar como apoio à decisão, e não como diagnóstico absoluto: uma mesma pontuação pode demandar respostas distintas conforme o tipo de dado comprometido e o contexto do usuário. O presente capítulo justifica a quantificação e seus critérios gerais, enquanto o Capítulo 4 detalha pesos, fatores temporais, normalização, faixas e validação do algoritmo.

2.4 Marco Regulatório (LGPD) e a Literacia Digital no Brasil

A Lei Geral de Proteção de Dados Pessoais (LGPD — Lei nº 13.709/2018; BRASIL, 2018) consolidou no Brasil princípios de transparência, prevenção e autodeterminação informativa. Entretanto, a existência do direito formal não assegura seu exercício material. Chen, Henry e Jiang (2023) sustentam que a simples divulgação de fatores de risco é insuficiente quando o titular não dispõe de uma estrutura para interpretar o impacto sobre sua

privacidade. Essa crítica aproxima o marco regulatório da discussão de Nemec Zlatolas, Welzer e Lhotska (2024): tanto a sobrecarga operacional quanto a opacidade informacional podem impedir respostas adequadas, ainda que o usuário tenha recebido um alerta.

A mediação tecnológica torna-se, portanto, condição para aproximar rigor normativo e prática cotidiana. Reuben-Owoh e Haig (2025) defendem que padrões de cibersegurança precisam ser acompanhados por mecanismos de comunicação personalizada. Essa proposição converge com Chen, Henry e Jiang (2023), mas acrescenta uma dimensão operacional: não basta informar; é necessário adequar a mensagem ao contexto decisório. Em contraponto, a inteligência de ameaças discutida por Conti, Dargahi e Dehghantanha (2018a) prioriza a qualidade e a correlação das evidências. A integração das duas vertentes indica que conformidade, análise técnica e literacia digital devem operar de forma articulada, sem que uma substitua a outra.

Nesse contexto, o SafeID é concebido como instrumento de apoio à conformidade e à cidadania digital, e não como substituto das obrigações dos agentes de tratamento. Ao combinar monitoramento, pontuação explicável e orientação em linguagem acessível, a proposta busca tornar o direito à informação operacional para o cidadão. Chen e Wang (2024) defendem que a gestão social de incidentes exige integrar resposta técnica e suporte educacional; em diálogo com Von Solms e Van Niekerk (2013), essa integração amplia a cibersegurança para além do perímetro organizacional e a situa no exercício cotidiano da autonomia informacional.

2.5 Síntese e Lacuna de Pesquisa

A literatura examinada oferece contribuições consistentes, porém fragmentadas:

4. Von Solms e Van Niekerk (2013) ampliam a cibersegurança para a governança pessoal;
5. Conti, Dargahi e Dehghantanha (2018a) estruturam a análise baseada em inteligência de ameaças;
6. Nemec Zlatolas, Welzer e Lhotska (2024) evidenciam as barreiras comportamentais à adoção de práticas protetivas;
7. Chen, Henry e Jiang (2023) e Chen e Wang (2024) demonstram a insuficiência da comunicação técnica sem mediação educacional;
8. o CVSS (FIRST, 2019) oferece um modelo de pontuação consolidado, mas voltado à severidade de vulnerabilidades.

Permanece pouco explorada, contudo, a integração desses eixos em uma solução voltada ao usuário comum brasileiro que correlacione exposições já documentadas, quantifique sua criticidade por um modelo transparente e explique prioridades de mitigação em linguagem acessível, sob os princípios da LGPD. Essa é a lacuna que o SafeID se propõe a preencher,

articulando detecção pós-incidente, *risk scoring* explicável e apoio cognitivo à decisão em uma única plataforma. Os objetivos da Seção 1.1 derivam diretamente dessa lacuna.

3 GESTÃO DO PROJETO

Este capítulo descreve a organização adotada para planejar, executar e acompanhar o desenvolvimento do SafeID. São apresentados a composição e as responsabilidades da equipe, as metodologias de gestão e desenvolvimento, os mecanismos de controle de riscos, a estrutura do repositório, o *backlog* do produto e o planejamento das entregas distribuídas entre as etapas do projeto.

3.1 Organização da Equipe

A equipe do SafeID é composta por quatro membros, distribuídos em funções que cobrem o ciclo completo de desenvolvimento, desde a concepção do produto até a implementação e persistência dos dados.

- **Matheus Pereira Pereira:** Product Owner e Scrum Master. Responsável pela visão do produto, gestão do *Backlog* e facilitação dos ritos ágeis.
- **Lucas Vieira de Souza:** Desenvolvedor Back-end. Responsável pela lógica de servidor, integração com APIs externas e inteligência artificial.
- **Joao Antônio Chamusca Martins:** Desenvolvedor Front-end. Responsável pela interface do usuário, responsividade e experiência do usuário (UX).
- **Kelvyn Gabriel Milu de Oliveira:** Desenvolvedor e Administrador de Banco de Dados (DBA). Responsável pela modelagem, integridade e performance das estruturas de dados.

3.1.1 Responsabilidades e Atividades

As atividades foram distribuídas de forma colaborativa por meio de uma matriz de responsabilidades (Quadro 2), que relaciona os papéis no Scrum, as especialidades técnicas, as principais entregas e a disponibilidade estimada de cada integrante. Para assegurar equilíbrio na participação, adotou-se uma dedicação uniforme de 10 a 12 horas semanais por membro, correspondente a 25% da capacidade total estimada da equipe. O PO/Scrum Master atua na interface com os requisitos de negócio, enquanto o time de desenvolvimento (DevTeam) concentra-se na implementação técnica e na integração coesa entre *Front-end*, *Back-end* e Banco de Dados.

Quadro 2 - Matriz de Responsabilidades e Atribuições da Equipe

Integrante	Papel no Scrum	Especialidade Técnica	Atribuições e Entregas	Dedicação Estimada
Matheus Pereira	Scrum Master / PO	Gestão e Dados	Liderança da documentação acadêmica, gestão do <i>Backlog</i> , curadoria dos dados das APIs e validação do cálculo de risco (<i>Score</i>).	10 a 12 h/semana (25%)
Kelvyn Gabriel	Desenvolvedor / DBA	Cibersegurança	Modelagem do banco de dados (PostgreSQL), segurança da arquitetura, definição dos pesos de criticidade dos vazamentos e conformidade com a LGPD.	10 a 12 h/semana (25%)
Lucas Vieira	Desenvolvedor	<i>Back-end</i> / APIs	Construção da API do SafeID (Node.js), integração com o <i>Have I Been Pwned</i> , conexão com a LLM (Inteligência Artificial) e lógica de processamento de metadados.	10 a 12 h/semana (25%)
João Antônio	Desenvolvedor	<i>Front-end</i> / UX	Design da interface (Figma), desenvolvimento dos componentes em React, implementação do termômetro de risco e validação da responsividade.	10 a 12 h/semana (25%)

3.1.1.1 Sinergia entre Artefatos e Documentação

A gestão do projeto foi desenhada para que a construção conceitual e a técnica caminhem juntas:

- **Documentação Coletiva:** Embora o Matheus lidere a redação acadêmica, Kelvyn, Lucas e João são responsáveis por documentar as especificações técnicas (Dicionário de Dados, Diagrama de Classes e Documentação da API no GitHub).
- **Repositório Único (Monorepo):** Optamos por um repositório centralizado no GitHub. Isso facilita o rastreamento de quem desenvolveu cada *feature* e contribui para que a documentação técnica esteja sempre próxima ao código (*README.md* e *Wiki*).
- **Integração de Conhecimentos:** A experiência profissional do Kelvyn em cibersegurança alimenta diretamente a lógica do Lucas nas APIs, enquanto o Matheus traduz essa complexidade técnica para a linguagem acessível da documentação e da interface projetada pelo João.

3.2 Metodologias de Gestão e Desenvolvimento

O projeto adota o **Scrum** como metodologia ágil de gestão, permitindo entregas incrementais e adaptação rápida a mudanças de requisitos durante o processo de construção.

3.2.1 Scrum (Metodologia Ágil)

O Scrum foi selecionado por sua eficácia em projetos de tecnologia onde a integração de diferentes camadas (IA, APIs e Banco de Dados) exige ciclos constantes de inspeção e adaptação.

3.2.1.1 Sprints

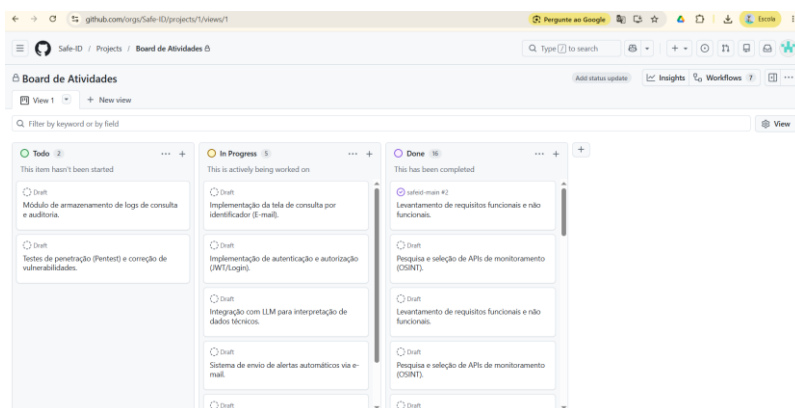
As atividades foram organizadas em ciclos quinzenais (*Sprints*), iniciados em março de 2026.

- **Planejamento:** No início de cada *Sprint*, a equipe define os itens do *Backlog* a serem desenvolvidos.
- **Construção e Documentação:** Paralelamente ao código, a documentação técnica é atualizada para garantir o registro da arquitetura.
- **Reuniões de Alinhamento:** Realizadas para identificar impedimentos técnicos, especialmente na integração com as APIs de monitoramento de incidentes.

O *Product Backlog*, o planejamento das *Sprints* e o acompanhamento das atividades são gerenciados no **GitHub Projects**, integrado ao repositório do SafeID. A ferramenta organiza os itens por estado, prioridade, responsável, fase e ciclo de entrega, além de relacionar tarefas a *issues*, *pull requests* e *commits*. Essa vinculação contribui para rastreabilidade entre requisitos,

responsabilidades, implementação, revisão e entrega, permitindo acompanhar o histórico de cada item sem alterar o conteúdo planejado para as Sprints. A Figura 1 apresenta o board de atividades utilizado no acompanhamento das Sprints.

Figura 1 - Board de Atividades do SafeID no GitHub Projects



Fonte: elaboração própria (2026). Disponível em: <https://github.com/orgs/Safe-ID/projects/1/views/1>.

3.2.2 Gestão de Riscos do Projeto

A gestão de riscos do SafeID é conduzida de forma contínua ao longo das *Sprints*, com identificação, avaliação qualitativa de probabilidade e impacto, definição de responsáveis e acompanhamento das ações de resposta no GitHub Projects. A priorização considera riscos capazes de comprometer a qualidade técnica, o cronograma, a disponibilidade da equipe ou serviços externos essenciais. Sempre que um risco se materializa, o item correspondente é reavaliado no planejamento da *Sprint* e recebe uma ação de contingência compatível com o impacto observado. Os principais riscos de execução, com probabilidade, impacto e resposta planejada, estão consolidados no Quadro 3.

Quadro 3 - Matriz de Riscos do Projeto SafeID

Categoria	Risco	Probabilidade / Impacto	Estratégia de Mitigação	Responsável	Data da última revisão
Técnico	Falhas na integração entre <i>Front-end</i> , <i>Back-end</i> , banco de dados	Média / Alta	Definir contratos de API, utilizar ambientes de homologação, testes	Lucas Vieira de Souza	24/09/2026

	e motor de risco.		automatizados de integração e integração contínua para detectar regressões antecipadamente.		
Prazo	Subestimação da complexidade ou acúmulo de itens críticos no fim da Sprint.	Média / Alta	Refinar o Backlog antes do planejamento, limitar trabalho em andamento, acompanhar marcos semanalmente e priorizar o caminho crítico e o MVP.	Matheus Pereira Pereira	24/09/2026
Equipe	Indisponibilidade e temporária de integrante ou concentração de conhecimento em uma única função.	Média / Média	Manter documentação técnica atualizada, realizar revisão cruzada, promover pareamento e compartilhar conhecimento sobre componentes críticos.	Matheus Pereira Pereira	24/09/2026

Dependência de terceiros	Indisponibilidade, limitação de taxa ou mudança de contrato da API <i>Have I Been Pwned</i> (HIBP).	Média / Alta	Aplicar <i>timeout</i> , retentativas com recuo exponencial, <i>Circuit Breaker</i> , cache de respostas válidas, filas assíncronas e mensagem de contingência; manter a camada de integração desacoplada para permitir fonte alternativa.	Lucas Vieira de Souza	24/09/2026
Técnico / Prazo	Atraso na integração da LLM ou inadequação das respostas geradas.	Média / Alta	Validar antecipadamente e uma prova de conceito, encapsular o provedor por interface, utilizar respostas padronizadas como <i>fallback</i> , testar <i>prompts</i> com casos controlados e	Lucas Vieira de Souza	24/09/2026

			preservar a entrega do diagnóstico determinístico sem dependência da LLM.		
--	--	--	---	--	--

As respostas previstas combinam prevenção, redução de probabilidade, limitação de impacto e contingência. Em especial, o desacoplamento de integrações externas evita que a indisponibilidade do HIBP ou o atraso da LLM interrompam as funções centrais do SafeID. A matriz deve ser revisada nas reuniões de alinhamento e atualizada quando houver alteração de probabilidade, impacto, responsável ou estratégia de tratamento.

3.3 Repositório da Aplicação

A gestão de configuração e versão do código-fonte é realizada por meio de ferramentas de controle de versão distribuídas.

3.3.1 Definição do Repositório

Foi adotado o **Git** como sistema de controle de versão, utilizando o **GitHub** como plataforma de hospedagem. Esta escolha justifica-se pela facilidade de colaboração entre a equipe e a integração contínua (CI/CD) com ferramentas de deploy.

3.3.2 Link do Repositório e Especificações para Acesso

O projeto **SafeID** adota o modelo de **Monorepo**, centralizando toda a inteligência de dados, documentação e código-fonte em um único repositório gerenciado dentro de uma organização no GitHub. Essa estrutura contribui para a rastreabilidade das entregas e a integridade entre o desenvolvimento técnico e a documentação acadêmica.

Organização dos Diretórios:

- **/client:** Contém o código-fonte da interface web desenvolvida em **React.js**.
- **/server:** Contém a lógica de negócio, integração com APIs OSINT e o motor de IA em **Node.js**.
- **/database:** Scripts de migração (DDL/DML), modelagem Entidade-Relacionamento e exemplos de esquemas de dados.
- **/docs:** Documentação oficial do projeto (incluindo o arquivo Word e versões em PDF), além de registros de requisitos.

- **/assets:** Diagramas de arquitetura, logotipos e recursos visuais do sistema.

Link de Acesso: <https://github.com/Safe-ID/safeid-main>

O repositório está configurado com um arquivo **README.md** na raiz que orienta a navegação e o processo de instalação. As entregas de cada *sprint* são evidenciadas através do histórico de *commits* e *logs* de atividades dos membros, permitindo a auditoria das contribuições individuais mencionadas no cronograma e validando a evolução do Produto Mínimo Viável (MVP).

A aplicação SafeID encontra-se disponível e operacional no endereço: <https://www.safeid.app.br>, podendo ser acessada publicamente para validação das funcionalidades implementadas.

3.3.3 Backlog do Produto (Product Backlog)

Seguindo as boas práticas no desenvolvimento de sistemas com a metodologia empregada, o *Backlog* do SafeID foi categorizado para permitir uma visão clara do que compõe o Protótipo (PI-1) e a Versão Final (PI-2). O Quadro 4 apresenta o *Backlog* do Produto, distinguindo os itens do Protótipo (PI-1) e da Versão Final (PI-2).

Quadro 4 - Backlog do Produto (Product Backlog - PI-1 e PI-2)

ID	Categoria	Descrição do Item	Fase	Prioridade
01	Requisito	Levantamento de requisitos funcionais e não funcionais.	PI-1	Essencial
02	Requisito	Pesquisa e seleção de APIs de monitoramento (OSINT).	PI-1	Essencial
03	Arquitetura	Definição da stack tecnológica (React, Node.js, DB).	PI-1	Essencial
04	Banco de Dados	Modelagem do Diagrama Entidade-Relacionamento (DER).	PI-1	Essencial
05	Banco de Dados	Implementação do esquema de tabelas para histórico e usuários.	PI-1	Alta
06	Front-end	Prototipação de alta fidelidade (Figma).	PI-1	Alta

07	<i>Back-end</i>	Desenvolvimento do servidor API (Node.js + NestJS)	PI-1	Essencial
08	Integração	Implementação do módulo de consumo da API externa (HIBP).	PI-1	Essencial
09	Lógica	Desenvolvimento do algoritmo de cálculo de <i>Risk Score</i> .	PI-1	Essencial
10	<i>Front-end</i>	Implementação da tela de consulta por identificador (E-mail).	PI-1	Alta
11	<i>Front-end</i>	Desenvolvimento do <i>Dashboard</i> de visualização de vazamentos.	PI-1	Alta
12	Documentação	Redação dos Capítulos 3 e 4 (Gestão e Análise).	PI-1	Essencial
13	Segurança	Implementação de autenticação e autorização (JWT/Login).	PI-2	Alta
14	Inteligência	Integração com LLM para interpretação de dados técnicos.	PI-2	Alta
15	Inteligência	Geração automática de plano de ação preventivo (via IA).	PI-2	Alta
16	<i>Back-end</i>	Módulo de armazenamento de logs de consulta e auditoria.	PI-2	Média
17	Segurança	Criptografia de dados sensíveis em repouso (LGPD).	PI-2	Essencial
18	<i>Front-end</i>	Implementação de temas (Dark/Light mode) e acessibilidade.	PI-2	Baixa
19	Notificação	Sistema de envio de alertas automáticos via e-mail.	PI-2	Média
20	Qualidade	Realização de testes de integração e carga.	PI-2	Alta
21	Segurança	Testes de penetração (Pentest) e correção de vulnerabilidades.	PI-2	Essencial
22	Documentação	Finalização da monografia e revisão para banca final.	PI-2	Essencial
23	Banco de Dados	Evolução do esquema e do painel para múltiplos identificadores por usuário (US07).	PI-2	Média

3.3.4 Detalhamento das Sprints (Projeto Integrado I - Março a Junho)

O cronograma de PI-1 foca na fundamentação teórica e na construção do protótipo funcional (MVP), cobrindo os itens **01 a 12** do Product Backlog.

Sprint 1: Concepção e Engenharia de Requisitos (Março)

- **Objetivo:** Estabelecer a base conceitual e técnica para o monitoramento de vazamentos.
- **Itens do Backlog:** 01, 02 e 12.
- **Artefatos:** Documento de Visão, Levantamento de Requisitos e Estudo de APIs (OSINT).
- **Entregas:** Definição do Escopo, Referencial Teórico e Configuração do Repositório.

Sprint 2: Modelagem e Design de Experiência (Abril)

- **Objetivo:** Traduzir os requisitos em arquitetura de dados e interface visual.
- **Itens do Backlog:** 03, 04, 05 e 06.
- **Artefatos:** DER (Diagrama Entidade-Relacionamento), Dicionário de Dados e Protótipo de Alta Fidelidade (Figma).
- **Entregas:** Arquitetura do Banco de Dados e Layout da Interface Homem-Máquina.

Sprint 3: Desenvolvimento do Core e Lógica de Risco (Maio)

- **Objetivo:** Implementar o motor de busca e o algoritmo de análise.
- **Itens do Backlog:** 07, 08 e 09.
- **Artefatos:** Servidor NestJS funcional, Módulo de integração HIBP e Algoritmo de cálculo de *Risk Score*.
- **Entregas:** *Back-end* operacional capaz de processar e categorizar incidentes reais.

Sprint 4: Integração Front-end e Entrega do Protótipo (Junho)

- **Objetivo:** Consolidar a Prova de Conceito (PoC) para apresentação à banca.
- **Itens do Backlog:** 10, 11 e 12.
- **Artefatos:** *Dashboard* em React integrado ao *Back-end* e Relatório Técnico Parcial.
- **Entregas:** Protótipo funcional com consulta e visualização de risco; Documentação de PI-1 finalizada.

3.3.5 Planejamento para Projeto Integrado II (Agosto a Dezembro)

Para o segundo semestre, o Roadmap foca na evolução do protótipo para um sistema mais completo, com integração de inteligência artificial e reforço de segurança (Itens **13 a 23**).

Agosto e Setembro: Autenticação e Inteligência Cognitiva

- **Foco:** Garantir a privacidade do usuário e a camada de IA.
- **Itens do Backlog:** 13, 14, 15, 16 e 23.

- **Atividades:** Implementação de autenticação JWT, integração com a API da LLM para interpretação dos vazamentos e geração do plano de ação preventivo personalizado, além da evolução do esquema de dados para múltiplos identificadores por usuário (US07).

Outubro: Conformidade, Segurança e Notificações

- **Foco:** Alinhamento com a LGPD e monitoramento ativo.
- **Itens do *Backlog*:** 17, 18 e 19.
- **Atividades:** Implementação de criptografia em repouso (banco de dados), desenvolvimento do sistema de alertas via e-mail e ajustes de acessibilidade e interface (Dark/Light mode).

Novembro: Garantia da Qualidade e Resiliência

- **Foco:** Estabilização do sistema para ambiente de produção.
- **Itens do *Backlog*:** 20 e 21.
- **Atividades:** Execução de testes de carga, testes de penetração (Pentest) para identificar vulnerabilidades e correções finais na lógica do sistema.

Dezembro: Encerramento e Defesa

- **Foco:** Consolidação dos resultados e apresentação final.
- **Item do *Backlog*:** 22.
- **Atividades:** Revisão ortográfica e técnica da monografia completa, preparação dos slides de defesa e entrega final do sistema SafeID funcional à banca examinadora.

4 DESENVOLVIMENTO DO PROJETO

Este capítulo apresenta a estrutura técnica e funcional adotada no desenvolvimento do SafeID, relacionando os requisitos definidos às decisões de implementação e validação do sistema. São detalhados o escopo, as regras de negócio, as histórias de usuário, a arquitetura, as tecnologias empregadas, a estratégia de testes, os controles de segurança e privacidade, o modelo de dados e os demais elementos necessários à consolidação da solução.

4.1 Escopo do projeto

O escopo do projeto **SafeID** compreende o desenvolvimento de uma aplicação web voltada à literacia e proteção de dados pessoais, atuando especificamente na camada de pós-incidente de segurança. A solução propõe-se a servir como uma ponte inteligível entre bases de dados técnicas de vazamentos (*data breaches*) e o usuário final, utilizando inteligência artificial para personalizar orientações de mitigação.

O que será desenvolvido (Inclusões):

- **Módulo de Consulta:** Interface para que o usuário submeta identificadores (como e-mail) para verificação em bases de dados OSINT (*Open Source Intelligence*), utilizando integração via API com provedores consolidados no mercado de cibersegurança.
- **Motor de Análise de Risco:** Algoritmo que processa a natureza do vazamento (ex: exposição de senhas, endereços IP, nomes completos) e atribui uma métrica de gravidade ponderada (*Risk Score*).
- **Assistente de Orientação (IA):** Integração com modelos de linguagem de larga escala (LLM) para traduzir metadados técnicos em planos de ação preventivos e compreensíveis para o público leigo.
- **Painel de Controle (Dashboard):** Visualização histórica das consultas realizadas e evolução do nível de exposição do usuário.

O que não será desenvolvido (Exclusões):

- **Recuperação de Contas:** O sistema não realizará ações diretas em contas de terceiros (ex: o SafeID não trocará a senha do e-mail do usuário automaticamente).
- **Monitoramento de Rede Local:** A aplicação não atuará como antivírus ou firewall residente no dispositivo do usuário.
- **Venda de Proteção Financeira:** O projeto não oferece seguros ou garantias financeiras contra fraudes, limitando-se à orientação e monitoramento informativo.

Premissas e Restrições da aplicação

- **Premissa:** O sistema depende da disponibilidade das APIs de terceiros (como *Have I Been Pwned*). Caso o serviço externo esteja offline, o SafeID apresentará uma mensagem de indisponibilidade momentânea, não sendo responsabilidade do grupo a manutenção de bases de dados de terceiros.
- **Restrição:** O sistema não realizará "quebra de *hashes*" ou descryptografia de senhas. Se o vazamento indicar que a senha está criptografada, o sistema apenas informará o status, cumprindo seu papel informativo e ético.

O projeto será entregue em duas etapas: no primeiro semestre, um **Protótipo Funcional (PoC)** para validar a comunicação com APIs e a lógica de risco; no segundo semestre, a **aplicação completa**, com interface finalizada, sistema de autenticação e assistente cognitivo integrado.

4.1.1 Regras de negócio

As regras de negócio a seguir definem as restrições e o funcionamento lógico do SafeID, assegurando que o sistema opere em conformidade com as diretrizes de cibersegurança e proteção de dados.

- **RN01 – Identificador Único de Consulta:** O sistema deve aceitar como identificador primário para consulta o endereço de e-mail. Para a fase de PI-2, o sistema poderá ser expandido para outros identificadores, desde que validados por máscaras específicas.
- **RN02 — Cálculo do *score* de risco:** w_i é o peso da classe de dado i ; D_j é o conjunto de classes expostas no incidente j ; $T(t_j)$ é o fator temporal. Para cada incidente j , $S_j = T(t_j) \times \sum_{i \in D_j} w_i$. O valor bruto é $S_{\text{bruto}} = \sum_{j=1}^N S_j$. O resultado normalizado é $\text{RiskScore} = \min(100, 100 \times S_{\text{bruto}}/S_{\text{max}})$, em escala contínua $[0,100]$, com $S_{\text{max}} = 50$.
- **RN03 — Fator temporal:** t_j é a idade do incidente j , expressa em meses. Define-se $T(t_j) = 1,0$ para $t_j \leq 24$; $T(t_j) = 0,7$ para $24 < t_j \leq 60$; e $T(t_j) = 0,4$ para $t_j > 60$.
- **RN04 – Limitação de Consultas (*Rate Limiting*):** Para evitar abusos e garantir a estabilidade da integração com APIs externas, cada usuário (ou IP não autenticado no protótipo) terá um limite de 3 consultas por hora.
- **RN05 – Anonimização de Resultados Sensíveis:** O sistema nunca deverá exibir a senha vazada em texto claro. Caso o metadado do vazamento inclua a senha, o SafeID deve apenas informar a existência da exposição e orientar a troca imediata, preservando a segurança do usuário mesmo dentro da plataforma.
- **RN06 – Personalização via IA:** O plano de ação gerado pela Inteligência Artificial deve ser obrigatoriamente contextualizado ao tipo de vazamento encontrado. Se o vazamento for de "Dados de Cartão", a recomendação prioritária deve ser o bloqueio bancário; se for de "Senha", a troca de credenciais e ativação de MFA.
- **RN07 – Histórico de Privacidade:** Consultas realizadas por usuários não autenticados (visitantes) não serão armazenadas no banco de dados após o encerramento da sessão, visando o princípio de minimização da LGPD. Somente usuários cadastrados terão seus históricos preservados para fins de acompanhamento evolutivo.

4.1.2 Requisitos funcionais

Os requisitos funcionais descrevem as funções que o software deve executar para atender às necessidades dos usuários e as regras de negócio estabelecidas. Para o SafeID, os requisitos foram divididos entre o que será entregue no protótipo (PI-1) e a implementação final (PI-2). Os requisitos funcionais (RF) estão listados no Quadro 5.

Quadro 5 - Requisitos Funcionais (RF)

ID	Requisito Funcional	Descrição	Fase
RF01	Consulta por Identificador (OSINT)	O sistema deve permitir a entrada de um endereço de e-mail e orquestrar, em <i>background</i> via filas distribuídas (BullMQ), a consulta à API <i>Have I Been Pwned</i> (HIBP), contribuindo para o respeito ao limite de requisições externas (<i>Rate Limit</i>).	PI-1
RF02	Categorização de Incidentes	O sistema deve processar o <i>payload</i> de retorno da API, extraindo e categorizando metadados estruturados, como nome da plataforma comprometida e tipologia dos dados vazados (ex: senhas em texto claro, IPs, <i>hashes</i>).	PI-1
RF03	Exibição do <i>Risk Score</i>	O sistema deve processar os dados expostos através do Motor de Risco interno, atribuindo pesos matemáticos de criticidade e gerando uma pontuação final (<i>Risk Score</i>), a ser exibida graficamente no <i>dashboard</i> do usuário.	PI-1
RF04	Tratamento de Erros e Validação	O sistema deve validar a entrada através de esquemas rígidos (<i>Class-validator</i> (NestJS)) e utilizar o padrão de resiliência (<i>Fail Fast / Circuit Breaker</i>) para informar ao usuário sobre e-mails inválidos, ausência de vazamentos ou indisponibilidade temporária das APIs.	PI-1
RF05	Geração de <i>Dashboard</i> Resumido	O <i>front-end</i> em React deve renderizar uma visualização consolidada, traduzindo as métricas brutas de risco e os incidentes técnicos em uma interface intuitiva e "Mobile First".	PI-1
RF06	Geração de Recomendações de Segurança Mitigatórias	O sistema deve processar os dados das chaves expostas e, por meio da integração com o modelo GPT-4o-mini , utilizando o SDK da OpenAI versão 6.41.0, gerar um conjunto de recomendações personalizadas para o usuário.	PI-2

RF07	Emissão de Diagnóstico de Risco Cognitivo	A aplicação deve utilizar inteligência artificial generativa com o modelo GPT-4o-mini , integrado por meio do SDK da OpenAI versão 6.41.0, para traduzir dados técnicos de vazamentos massivos em relatórios simples, legíveis e didáticos para o usuário final de perfil B2C.	PI-2
RF08	Gestão de Contas de Usuário	O sistema deve prover um módulo de autenticação, permitindo o armazenamento seguro (via PostgreSQL/Prisma) do histórico de consultas vinculadas ao perfil do usuário, aplicando criptografia em repouso.	PI-2
RF09	Monitoramento Ativo de Ameaças	O ecossistema deve orquestrar <i>Background Jobs</i> recorrentes para consultar silenciosamente as bases de dados públicas, disparando alertas proativos ao usuário cadastrado em caso de novas exposições detectadas.	PI-2
RF10	Exportação de Relatório (PDF)	O sistema deve permitir a consolidação do <i>Dashboard</i> e dos planos de ação gerados pela IA em um documento portátil e padronizado (PDF), viabilizando a consulta <i>offline</i> .	PI-2

4.1.3 Requisitos não funcionais

Enquanto os funcionais dizem *o que* o sistema faz, os não funcionais descrevem *como* ele deve ser (qualidade, segurança e performance). Os requisitos não funcionais (RNF) estão listados no Quadro 6.

Quadro 6 - Requisitos Não Funcionais (RNF)

ID	Requisito Não Funcional	Categoria	Descrição
-----------	--------------------------------	------------------	------------------

RNF01	Privacidade por Design (LGPD)	Segurança	O sistema não deve trafegar ou armazenar e-mails consultados em texto claro para serviços de <i>cache</i> , utilizando técnicas de ofuscação (<i>Hash</i> SHA-256) no Redis e criptografia simétrica em repouso feita na aplicação, campo <i>String</i> (AES-256-GCM) no banco PostgreSQL.
RNF02	Desempenho e Orquestração	Desempenho	O sistema deve otimizar o tempo de resposta priorizando o <i>cache</i> de memória (Redis). Para consultas novas, deve respeitar os limites rígidos de requisições da API externa (HIBP) utilizando filas distribuídas (BullMQ).
RNF03	Integração com o Motor Cognitivo	Usabilidade	O ecossistema de inteligência artificial deve utilizar o modelo GPT-4o-mini , obrigatoriamente intermediado pelo SDK oficial da OpenAI versão 6.41.0, estabelecendo como meta que o tempo de resposta percebido pelo usuário, desde a solicitação até a entrega do diagnóstico cognitivo, não ultrapasse 3 segundos.
RNF04	Resiliência e Tolerância a Falhas	Confiabilidade	A aplicação deve implementar o padrão <i>Circuit Breaker</i> (Opossum) para cortar a comunicação com APIs externas caso a taxa de erro ultrapasse 50%, prevenindo lentidão em cascata e exaustão de recursos do servidor.
RNF05	Compatibilidade e Responsividade	Portabilidade	A interface <i>front-end</i> deve adotar a abordagem "Mobile First", sendo responsiva e compatível com

			navegadores desktop e dispositivos móveis atuais.
RNF06	Segurança de Tráfego e API	Segurança	O Helmet reduz a exposição associada a cabeçalhos HTTP e aplica políticas de segurança configuráveis, mas não previne sozinho XSS; sua eficácia depende da configuração e de controles complementares de validação, codificação de saída e CSP.
RNF07	Integridade e Sanitização de Dados	Segurança	Toda entrada de dados recebida pela API deve passar por um funil definido de validação e tipagem (utilizando esquemas <i>Class-validator</i> (NestJS)) para garantir o formato RFC de e-mails e prevenir vulnerabilidades como Buffer Overflow e injeções.
RNF08	Manutenibilidade e Qualidade (DevOps)	Manutenibilidade	O código-fonte deve manter uma cobertura de testes de unidade (Jest) superior a 80%, sendo validado automaticamente por esteiras de CI/CD (GitHub Actions) e empacotado para produção utilizando contêineres (<i>Docker Multi-stage Build</i>).

4.1.4 Modelo de Avaliação de Risco e Ponderação de Exposição (Risk Score)

A quantificação do risco de segurança e exposição digital no SafeID fundamenta-se na necessidade de converter metadados heterogêneos de incidentes cibernéticos (*Open Source Intelligence - OSINT*) em uma métrica unificada, intuitiva e acionável para o cidadão leigo. A ausência de padronização na comunicação de vazamentos frequentemente gera passividade ou alarmismo desproporcional; assim, o modelo matemático do SafeID estabelece um rigor técnico e normativo para a avaliação de danos potenciais.

4.1.4.1 Fundamentação Teórica e Normativa

A atribuição dos pesos de criticidade e a arquitetura da equação matemática baseiam-se em quatro pilares do ecossistema de cibersegurança e proteção de dados:

1. **Diretrizes de Identidade Digital e Autenticação (NIST SP 800-63B):** O National Institute of Standards and Technology (NIST, 2017) estabelece que segredos de autenticação (como senhas em texto claro ou *hashes* de baixa entropia) possuem o maior potencial de comprometimento de contas através de ataques automatizados de *Credential Stuffing* e *Password Spraying*. Portanto, essas categorias recebem o peso máximo de criticidade.
2. **Métricas de Impacto de Vulnerabilidades (CVSS v3.1 / v4.0):** Inspirando-se no *Common Vulnerability Scoring System (FIRST, 2019)*, a taxonomia do SafeID avalia os vetores de confidencialidade e potencial de exploração secundária. Atributos que permitem fraudes financeiras diretas ou usurpação de identidade legal possuem peso superior a metadados puramente técnicos (como o endereço IP).
3. **Lei Geral de Proteção de Dados (LGPD - Lei nº 13.709/2018):** Com base no Art. 5º, incisos I e II, e no Art. 52 da Lei (BRASIL, 2018), a classificação diferencia dados pessoais comuns de dados sensíveis ou cadastrais de alto impacto, como CPF, dados financeiros e dados biométricos, alinhando a ponderação à severidade do dano potencial ao titular.
4. **Literatura Científica em Threat Intelligence:** Conforme demonstrado por Conti, Dargahi e Dehghantanha (2018a), dados exfiltrados alimentam mercados secundários de cibercrime. Adicionalmente, Nemec Zlatolas, Welzer e Lhotska (2024) observam que a reutilização de senhas torna os vazamentos recentes exponencialmente mais perigosos, justificando a incorporação de um fator de atenuação temporal (*T*) na modelagem.

4.1.4.2 Taxonomia e Tabela de Pesos por Categoria de Dado (Data Classes)

Os metadados retornados pelas fontes de inteligência de ameaças e *Open Source Intelligence* (OSINT — Inteligência de Fontes Abertas) são mapeados e categorizados em quatro níveis de criticidade para o cálculo do risco, conforme a Tabela 1:

Tabela 1 - Categorias e pesos dos metadados obtidos da API

Nível de Criticidade	Classe de Dado (<i>Data Class</i>)	Peso (w_i)	Justificativa Teórica e Regulatória
Nível 4: Altíssima	Dados Financeiros (Cartão de Crédito/Débito, Card Verification Value — CVV, Dados Bancários, Chave de Pagamento Instantâneo — PIX)	10	Risco imediato de dano patrimonial direto e fraude financeira automatizada (Lei Geral de Proteção de Dados Pessoais — LGPD, Lei nº 13.709/2018, art. 52).
Nível 4: Altíssima	Senhas em Texto Claro (<i>Plaintext Passwords</i>) ou <i>Hashes</i> de Baixa Entropia (<i>Message Digest Algorithm 5</i> — MD5, <i>Secure Hash Algorithm 1</i> — SHA-1)	10	Permite acesso direto e não autorizado às contas do usuário por ataques automatizados de <i>credential stuffing</i> (National Institute of Standards and Technology — NIST, Special Publication 800-63B).
Nível 3: Alta	<i>Hashes</i> de Senha de Alta Entropia (Bcrypt, Argon2, <i>Password-Based Key Derivation Function 2</i> — PBKDF2)	7	Exige esforço computacional elevado para quebra <i>offline</i> (<i>offline cracking</i>), mas ainda representa risco grave de exposição da credencial.
Nível 3: Alta	Documentos Oficiais Pessoais (Cadastro de Pessoas Físicas — CPF, Registro Geral — RG, Passaporte, Carteira Nacional de Habilitação — CNH)	8	Permite personificação ilícita, abertura de contas fraudulentas e falsidade ideológica em serviços terceiros.
Nível 3: Alta	Tokens de Autenticação, Chaves de <i>Multi-Factor Authentication</i> (MFA — Autenticação de Múltiplos Fatores) e Biometria	8	Comprometimento do segundo fator de autenticação e quebra de barreiras de proteção avançada do usuário.

Nível 2: Média	Número de Telefone Residencial/Móvel e Endereço Residencial Físico	5	Facilita ataques direcionados de engenharia social, troca fraudulenta de chip <i>Subscriber Identity Module</i> (SIM swapping) e <i>phishing</i> via <i>Short Message Service</i> (SMS) ou mensageiros.
Nível 2: Média	Data de Nascimento e Filiação Completa	4	Dados utilizados frequentemente em perguntas de validação de segurança e processos cadastrais secundários.
Nível 1: Baixa	Nome de Usuário (<i>Username</i>) e Nome Completo do Titular	3	Informação de identificação pública ou semipública; isoladamente possui baixo potencial ofensivo direto.
Nível 1: Baixa	Endereço de <i>Internet Protocol</i> (IP) e Dados de Geolocalização Aproximada	2	Metadado técnico de conexão de rede que se altera com frequência; risco limitado de exploração individual direta.
Nível 1: Baixa	<i>User-Agent</i> e Metadados Técnicos de Navegador e Dispositivo	1	Telemetria técnica básica de infraestrutura sem vínculo direto com credenciais privadas de acesso.

4.1.4.3 Modelagem Matemática do Algoritmo

Seja N o número total de incidentes de segurança (*data breaches*) nos quais o identificador do usuário foi localizado. Para cada incidente j pertencente ao conjunto $\{1, 2, \dots, N\}$, define-se D_j como o conjunto de categorias de dados comprometidos.

1. Fator de Atenuação Temporal $T(t_j)$

Dado o tempo decorrido t_j , em meses, desde a data de publicação ou ocorrência do incidente j , o coeficiente de relevância temporal $T(t_j)$ é definido pela função degrau:

$$T(t_j) = 1,0, \text{ se } t_j \leq 24 \text{ meses}; 0,7, \text{ se } 24 < t_j \leq 60 \text{ meses}; 0,4, \text{ se } t_j > 60 \text{ meses}.$$

2. Pontuação do Incidente S_j

A pontuação de severidade do incidente j corresponde à soma dos pesos das categorias expostas, ajustada pelo fator temporal:

$$S_j = T(t_j) \cdot \sum_{i \in D_j} w_i$$

Onde w_i representa o peso atribuído à classe de dado i .

3. Pontuação Bruta Acumulada S_{bruto}

O somatório do impacto de todos os vazamentos associados ao titular é calculado por:

$$S_{\text{bruto}} = \sum_{j=1}^N S_j$$

4. Normalização e Obtenção do *Risk Score* Final

Para mapear o resultado em um índice contínuo de 0 a 100, utiliza-se uma função de saturação linear com limite $S_{\text{max}} = 50$:

$$\text{RiskScore} = \min(100, (S_{\text{bruto}} / S_{\text{max}}) \cdot 100), \text{ com } S_{\text{max}} = 50$$

A escolha de $S_{\text{max}} = 50$ resulta de uma análise de sensibilidade determinística com $S_{\text{max}} \in \{30, 40, 50, 60, 75\}$. Enquanto $S_{\text{bruto}} < 50$, cada ponto bruto corresponde a 2 pontos na escala final: uma contribuição crítica recente de peso 10 gera score 20, e duas contribuições equivalentes geram 40; a saturação em 100 ocorre apenas quando a soma ajustada alcança 50. Limites 30 e 40 saturariam o modelo após, respectivamente, três e quatro contribuições de peso 10, ao passo que 50 preserva discriminação por uma contribuição crítica adicional. Limites 60 e 75, por outro lado, comprimem os níveis intermediários. Trata-se de análise baseada em cenários do próprio modelo, não de evidência empírica de produção.

5. Faixas de Mapeamento do Termômetro de Risco

- De 0 a 25: Nível de Risco Baixo (cor verde na interface) — ação recomendada: manutenção de boas práticas e monitoramento passivo.
- De 26 a 50: Nível de Risco Médio (cor amarela na interface) — ação recomendada: atualização preventiva de senhas antigas e ativação de *Multi-Factor Authentication* (MFA — Autenticação de Múltiplos Fatores).
- De 51 a 75: Nível de Risco Alto (cor laranja na interface) — ação recomendada: troca imediata de credenciais e verificação de contas reutilizadas.
- De 76 a 100: Nível de Risco Crítico (cor vermelha na interface) — ação recomendada pela Inteligência Artificial (Requisito Funcional RF06): bloqueio prévio de cartões, auditoria de extratos e revogação imediata de acessos em aplicações vinculadas.

4.1.4.4 Exemplo Numérico Prático e Auditável

Para fins de validação técnica e verificação de auditoria, considere a simulação de consulta de um e-mail cadastrado localizado em três incidentes de segurança (*data breaches*).

Detalhamento dos Incidentes Encontrados

1. Incidente 1 — Vazamento no comércio eletrônico “LojaX”:

- Data da ocorrência: há 6 meses ($t_1 = 6$ meses; $T(t_1) = 1,0$).
- Dados expostos: senha em texto claro ($w = 10$), endereço de *Internet Protocol* (IP; $w = 2$) e nome completo ($w = 3$).

2. Incidente 2 — Vazamento no fórum “ServiçoY”:

- Data da ocorrência: há 36 meses ($t_2 = 36$ meses; $T(t_2) = 0,7$).
- Dados expostos: *hash* de senha Bcrypt ($w = 7$) e nome de usuário (*username*; $w = 3$).

3. Incidente 3 — Vazamento no aplicativo “PlataformaZ”:

- Data da ocorrência: há 12 meses ($t_3 = 12$ meses; $T(t_3) = 1,0$).
- Dados expostos: cartão de crédito/dados financeiros ($w = 10$) e número de telefone ($w = 5$).

Execução do Cálculo Passo a Passo

Cálculo do Incidente 1 (S_1):

$$\Sigma(i \in D_1) w_i = 10 + 2 + 3 = 15$$

$$S_1 = 1,0 \cdot 15 = 15,0$$

Cálculo do Incidente 2 (S_2):

$$\Sigma(i \in D_2) w_i = 7 + 3 = 10$$

$$S_2 = 0,7 \cdot 10 = 7,0$$

Cálculo do Incidente 3 (S_3):

$$\Sigma(i \in D_3) w_i = 10 + 5 = 15$$

$$S_3 = 1,0 \cdot 15 = 15,0$$

Somatório da Pontuação Bruta (S_{bruto}):

$$S_{\text{bruto}} = S_1 + S_2 + S_3 = 15,0 + 7,0 + 15,0 = 37,0 \text{ pontos}$$

Normalização para o *Risk Score* Final:

$$\text{RiskScore} = \min(100, (37,0 / 50) \cdot 100) = \min(100, 74) = 74$$

Resultado do Diagnóstico do Sistema

- **Score calculado:** 74/100.
- **Classificação de risco:** risco alto, representado em cor laranja no painel (*dashboard*).
- **Interpretação do diagnóstico:** embora o vazamento no fórum antigo tenha sofrido atenuação temporal, reduzindo seu peso de 10 para 7,0 pontos, a combinação de uma senha exposta em texto claro em incidente recente com o comprometimento de dados

bancários eleva o risco individual para o nível alto, demandando alteração imediata de credenciais e bloqueio de cartões.

4.2 Histórias de usuário

Esta seção apresenta as histórias de usuário elaboradas para representar as necessidades funcionais dos diferentes perfis que interagem com o SafeID. A subseção seguinte descreve cada história e seus respectivos critérios de aceite, estabelecendo a correspondência entre as expectativas dos usuários e os comportamentos previstos para o sistema.

4.2.1 Descrição das Histórias de Usuário

US01 – Consulta de Vulnerabilidade (PI-1)

- História: Como cidadão preocupado com minha segurança, eu quero inserir meu e-mail principal no sistema para verificar se minhas informações foram expostas em vazamentos de dados conhecidos.
- Critérios de Aceite:
 - O sistema deve validar o formato do e-mail inserido utilizando esquemas de validação estrita (ex: *Class-validator* (NestJS)).
 - A requisição deve ser enfileirada (via BullMQ) para respeitar os limites de taxa da API externa, exibindo um *feedback* visual (*loading*) ao usuário.
 - Em caso de indisponibilidade da API externa, o sistema deve acionar o *Circuit Breaker* e exibir uma mensagem amigável de erro.

US02 – Entendimento do Nível de Risco (PI-1)

- História: Como usuário leigo em tecnologia, eu quero visualizar um índice de risco (*score*) de fácil compreensão para que eu saiba a urgência de tomar medidas de proteção.
- Critérios de Aceite:
 - O *score* deve ser apresentado de forma visual (ex: termômetro ou cores de semáforo).
 - O valor numérico deve ser processado pelo Motor de Risco interno, classificando os dados expostos em níveis de criticidade (1 a 4).
 - Deve haver uma breve legenda explicando o que o nível "Baixo", "Médio" ou "Alto" significa.

US03 – Tradução de Metadados via IA (PI-2)

- História: Como usuário sem conhecimento técnico, eu quero que a inteligência artificial explique em linguagem simples o que significa cada termo técnico do vazamento (ex: "SQL Injection" ou "Combolist") para que eu entenda o impacto real na minha privacidade.

- Critérios de Aceite:
 - O sistema deve enviar à LLM (modelo GPT-4o-mini, acessado pelo SDK oficial da OpenAI, versão 6.41.0) apenas os metadados do vazamento, sem enviar o e-mail real do usuário, preservando a privacidade.
 - O texto deve evitar jargões complexos, focando na consequência prática para o usuário.

US04 – Plano de Ação Preventivo (PI-2)

- História: Como vítima de uma exposição de dados, eu quero receber orientações passo a passo do que fazer para mitigar o dano, para que eu possa proteger minhas contas de forma assertiva.
- Critérios de Aceite:
 - As recomendações devem ser dinâmicas e baseadas na resposta da IA sobre os dados vazados.
 - O plano deve incluir links ou tutoriais simples sobre como ativar a autenticação em duas etapas (MFA).

US05 – Monitoramento de Histórico (PI-2)

- História: Como usuário cadastrado, eu quero salvar meu histórico de consultas para que eu possa acompanhar se as medidas que tomei reduziram meu *score* de vulnerabilidade ao longo do tempo.
- Critérios de Aceite:
 - O sistema deve exigir login seguro para acessar o histórico.
 - O histórico deve ser recuperado do banco de dados (PostgreSQL)criptografando os registros salvos com AES-256-GCM.
 - O *Dashboard* deve exibir um gráfico comparativo entre a primeira consulta e a situação atual.

US06 – Autenticação e Privacidade (PI-2)

- História: Como usuário recorrente, eu quero criar uma conta protegida para que minhas consultas e dados de histórico não fiquem expostos a terceiros que utilizem o mesmo dispositivo.
- Critérios de Aceite:
 - O sistema deve permitir o cadastro e validar o e-mail em conformidade com as normas de higienização de dados.
 - As senhas devem ser salvas utilizando *hash* irreversível.

- O acesso deve ser gerido via tokens seguros (JWT), sendo interrompido após inatividade.

US07 – Gestão de Múltiplos Identificadores (PI-2)

- História: Como responsável pela segurança digital da minha família, eu quero cadastrar múltiplos e-mails no meu painel para monitorar a vulnerabilidade de todos os meus dependentes em um único lugar.
- Critérios de Aceite:
 - O usuário deve conseguir adicionar e remover e-mails da sua lista de monitoramento.
 - O *dashboard* deve separar os resultados por identificador.

US08 – Notificações de Alerta Ativo (PI-2)

- História: Como usuário preventivo, eu quero ser notificado via e-mail assim que um novo vazamento contendo meus dados for detectado, para que eu possa agir antes que ocorra uma invasão real.
- Critérios de Aceite:
 - O sistema deve orquestrar *Background Jobs* para realizar varreduras automáticas periódicas silenciosas.
 - O alerta deve conter um link direto para o plano de ação sugerido.

US09 – Exportação de Plano de Mitigação (PI-2)

- História: Como usuário que deseja se organizar offline, eu quero exportar o plano de ação gerado pela IA em formato PDF para que eu possa segui-lo como um checklist de segurança.
- Critérios de Aceite:
 - O documento gerado deve conter o resumo do vazamento e as instruções passo a passo.
 - O arquivo deve ser leve e formatado adequadamente para leitura.

US10 – Administração e Auditoria (PI-2)

- História: Como administrador do sistema, eu quero visualizar métricas agregadas (anonimizadas) de consultas para entender quais são os vazamentos mais comuns e treinar melhor o modelo de IA.
- Critérios de Aceite:
 - O painel administrativo deve exibir o volume de consultas por período.

- O sistema deve aplicar ofuscação (ex: *Hash* SHA-256) em informações analíticas, estabelecendo como meta que nenhum e-mail identificável seja exposto nesta visão (*Privacy by Design*).

4.3 Arquitetura

Esta seção descreve a arquitetura definida para o SafeID e os princípios que orientam a organização de seus componentes. Inicialmente, são apresentadas as decisões estruturais relativas às camadas de apresentação, processamento, integração, persistência e *cache*; posteriormente, o diagrama de arquitetura sintetiza visualmente as relações entre esses elementos.

4.3.1 Definições da arquitetura

O SafeID adota uma arquitetura de Monólito Modular (Modular Monolith). Diferentemente de uma arquitetura de microsserviços puros — que introduziria complexidade desnecessária de orquestração e rede (over-engineering) para o escopo deste MVP —, o monólito modular mantém a aplicação em uma base de código unificada através do *framework* NestJS, mas com limites de domínio estritamente separados. O desempenho mensurável e o isolamento de falhas não ocorrem pela fragmentação física da aplicação, mas sim pelo processamento assíncrono: tarefas de alta latência e operações I/O-bound, como a consulta à API OSINT externa e a inferência de Inteligência Artificial, são desacopladas da thread principal e enviadas para *workers* de processamento via filas (Redis e BullMQ). Essa abordagem entrega a capacidade técnica de processamento necessária, mantendo a simplicidade operacional e de implantação de um monólito.

- **Front-end (React.js & Tailwind CSS):** Camada de apresentação focada em prover uma experiência de usuário (UX) consistente, intuitiva e acessível para o público B2C. Utiliza a abordagem *Mobile-First* para garantir total responsividade entre dispositivos móveis e desktops. O Tailwind CSS é empregado para assegurar a agilidade no desenvolvimento e a consistência da identidade visual da marca. Toda a comunicação com o servidor ocorre de forma estritamente assíncrona por meio de requisições HTTPS.
- **Back-end e serviços externos (Node.js, TypeScript, NestJS, HIBP e OpenAI):** O *back-end* concentra o Motor de Risco e organiza as regras por módulos. DTOs e *class-validator* validam entradas; o Helmet aplica políticas de cabeçalhos que reduzem exposição técnica; Redis e BullMQ cadenciam tarefas em segundo plano; e o *Circuit Breaker*, implementado com Opossum, limita o efeito de falhas de serviços externos.

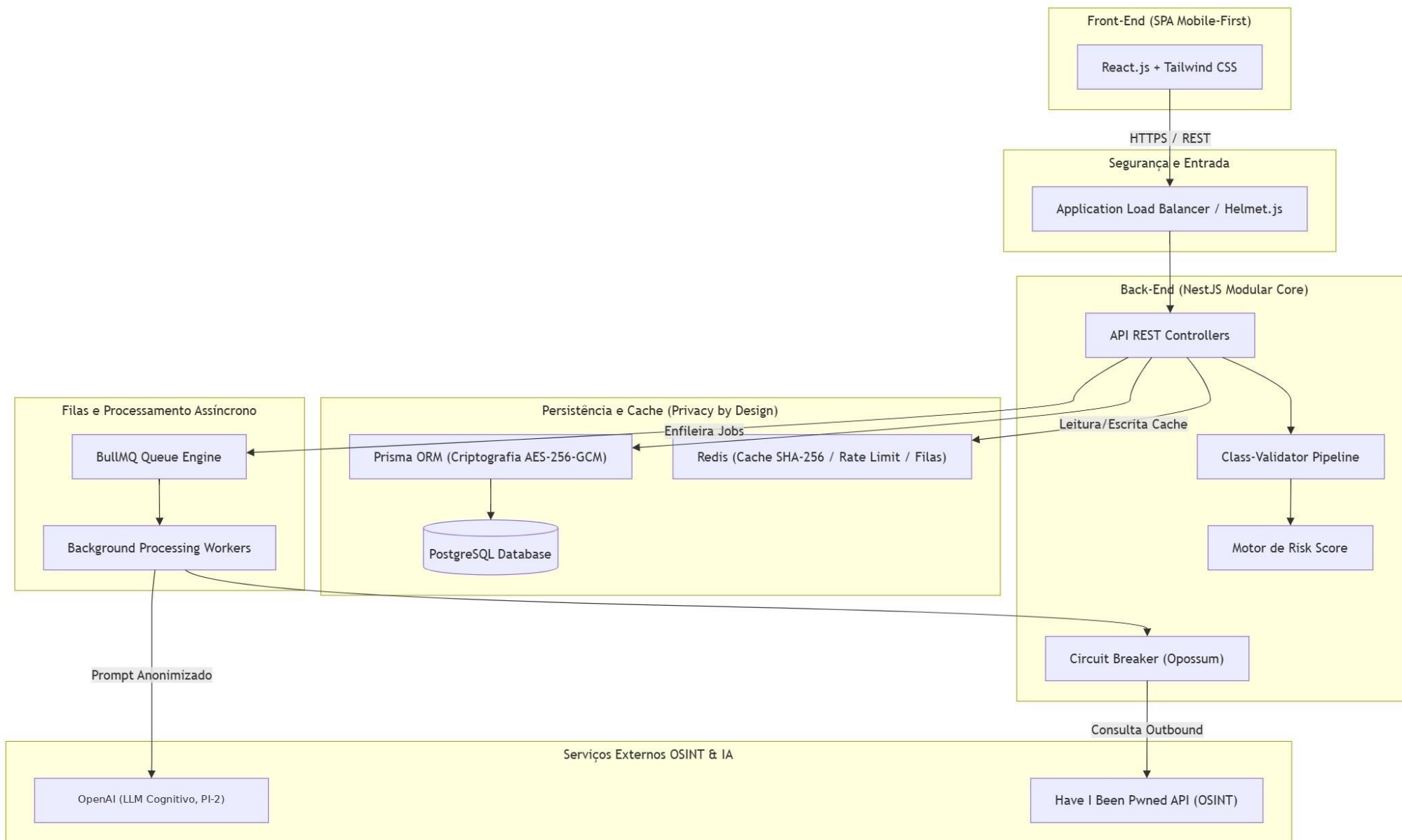
As integrações com HIBP e OpenAI são mantidas em interfaces separadas para favorecer testes e substituição de provedores.

- **Persistência e *Cache* (PostgreSQL, Prisma ORM e Redis):** A camada de dados foi projetada segundo o princípio de *Privacy by Design* e em alinhamento com as diretrizes da LGPD, separando armazenamento temporário em memória e persistência relacional.
 1. *Persistência Volátil (Redis)*: Atua como *cache* de consultas repetidas e suporte às filas. Os identificadores são pseudonimizados ou ofuscados com SHA-256; essa medida não garante anonimização e depende de controles de acesso e, quando aplicável, de sal ou chave.
 2. *Persistência Relacional (PostgreSQL e Prisma ORM)*: Armazena o histórico de monitoramento previsto para PI-2. O AES-256-GCM é aplicado no nível da aplicação e protege confidencialidade e integridade quando chaves, vetores de inicialização e etiquetas de autenticação são geridos corretamente.

A Figura 2, apresentada na Seção 4.3.2, detalha essa arquitetura em camadas: apresentação, segurança de entrada, processamento no *back-end*, filas, persistência e *cache*, e serviços externos.

4.3.2 Diagrama da arquitetura

Figura 2 - Diagrama de arquitetura do SafeID



4.4 Tecnologias

Esta seção apresenta as tecnologias selecionadas para a implementação e a operação do SafeID, considerando critérios de adequação ao escopo, segurança, desempenho e manutenibilidade. As subseções detalham as soluções adotadas no *front-end*, no *back-end*, na persistência de dados, na integração com inteligência artificial e serviços externos, bem como na infraestrutura em nuvem prevista para a aplicação.

4.4.1 Front-end: React.js e Tailwind CSS

A interface do SafeID é concebida como o componente responsável por converter dados complexos em informações acionáveis pelo cidadão. Para viabilizar essa entrega, utiliza-se a biblioteca React.js (META, [s. d.]), fundamentada em uma arquitetura baseada em componentes. Em comparação com Vue, o React oferece maior aderência ao ecossistema já adotado pela equipe e ampla disponibilidade de bibliotecas para gerenciamento de estado e testes; em relação ao Angular, reduz a imposição de uma estrutura completa de *framework*, adequada ao escopo incremental do MVP. A escolha também se apoia na eficiência do seu algoritmo de reconciliação (*Virtual DOM*), que permite atualizações granulares na interface sem a necessidade de recarregar a página integralmente. No contexto do SafeID, o React.js gerencia o estado de requisições assíncronas complexas — como o tempo de espera (*loading*) enquanto o *back-end* processa filas no BullMQ ou a exibição de *fallbacks* caso o *Circuit Breaker* seja acionado. Ao decompor a interface em unidades lógicas, elementos críticos, como o *Dashboard* de vulnerabilidades e o indicador de *Risk Score*, funcionam de forma isolada e testável.

Integrado ao React, o Tailwind CSS (TAILWIND LABS, [s. d.]) gerencia a estilização pelo paradigma *utility-first*. A abordagem *Mobile-First* favorece responsividade e acessibilidade em diferentes dispositivos. As classes utilitárias também permitem representar visualmente as faixas do Motor de Risco e controlar o volume dos artefatos de estilo; o efeito sobre métricas de carregamento deve ser verificado por testes de desempenho.

A combinação entre React e Tailwind CSS permite separar componentes funcionais e regras de apresentação, mantendo consistência visual no consumo de APIs e na exibição dos resultados. A camada de apresentação renderiza as explicações geradas pela Inteligência Artificial em formato didático e acessível, reduzindo a assimetria de informação entre especialistas em cibersegurança e o público leigo. Dessa forma, a interface contribui para a transparência e a clareza exigidas pelas diretrizes da LGPD.

4.4.2 Back-end: Node.js, NestJS e Prisma ORM

A infraestrutura de serviços do SafeID, responsável por orquestrar a lógica de processamento e a comunicação com as bases de inteligência cibernética, foi desenvolvida sobre o ecossistema Node.js (NODE.JS, [s. d.]) utilizando TypeScript em modo estrito. A escolha desta *stack* fundamenta-se no modelo de I/O não bloqueante e orientado a eventos do Node.js, que permite ao servidor gerenciar múltiplas requisições assíncronas, como as chamadas à API OSINT e as requisições de análise cognitiva ao modelo GPT-4o-mini, integrado por meio do SDK da OpenAI versão 6.41.0. Para favorecer manutenibilidade, testabilidade e isolamento das regras de negócio, o *back-end* foi estruturado com o **NestJS**. Em comparação com o Express puro, o NestJS fornece organização modular, injeção de dependência e convenções arquiteturais integradas, reduzindo a necessidade de definir manualmente esses mecanismos no projeto. O *framework* organiza o sistema por meio de módulos, *controllers* e *providers* (serviços), implementando os princípios de Inversão de Controle (IoC) e Injeção de Dependência (DI).

O NestJS fornece a camada HTTP e o roteamento REST, além de interceptadores, *guards* e *middlewares*. DTOs e *class-validator* aplicam validação declarativa; o Helmet reduz a exposição de cabeçalhos e permite configurar políticas de segurança; Redis e BullMQ organizam filas em segundo plano; e o *Circuit Breaker*, com Opossum (NODESHIFT, 2026), limita a propagação de falhas de serviços externos. Esses controles reduzem riscos específicos, mas dependem de configuração, testes e monitoramento.

Na persistência, o Prisma ORM (PRISMA, [s. d.]) oferece uma abstração orientada a tipos e utiliza consultas parametrizadas, o que reduz a exposição a injeção SQL sem eliminar a necessidade de validação e controle de acesso. Em alinhamento com *Privacy by Design*, o *cache* emprega SHA-256 como pseudonimização dos identificadores, e os dados sensíveis persistidos recebem criptografia AES-256-GCM no nível da aplicação, cuja eficácia depende da gestão adequada de chaves, vetores de inicialização e etiquetas de autenticação.

A combinação entre o modelo assíncrono do Node.js, a modularidade do NestJS e a tipagem do Prisma ORM fornece uma base adequada ao processamento e à mitigação de riscos previstos no escopo. Essa estrutura favorece a separação de responsabilidades, a realização de testes e a evolução controlada dos módulos, mantendo a viabilidade técnica do produto no contexto acadêmico e social proposto.

4.4.3 Banco de Dados: PostgreSQL

A camada de armazenamento do SafeID foi projetada segundo o princípio de *Privacy by Design*, com estratégia híbrida de persistência relacional e em memória destinada a favorecer desempenho mensurável, segurança e alinhamento com as diretrizes da LGPD.

A persistência de longo prazo é estruturada sobre o sistema de gerenciamento de banco de dados relacional PostgreSQL (POSTGRES GLOBAL DEVELOPMENT GROUP, [s. d.]), escolhido pela integridade transacional, pelo suporte a restrições relacionais e pelos recursos de segurança. Em comparação com o MongoDB, o PostgreSQL é mais aderente às relações entre usuários, históricos e registros de auditoria do SafeID, pois permite impor consistência referencial e transações ACID diretamente no esquema.

Limitação e risco técnico: uma instância relacional central pode representar ponto único de falha caso não haja redundância, replicação e rotinas de recuperação verificadas. A mitigação exige *backups* testados, monitoramento, restauração documentada e, no ambiente de produção, configuração de alta disponibilidade.

Academicamente, o PostgreSQL justifica-se por ser um banco de dados de código aberto que implementa de forma sistemática os padrões SQL, oferecendo suporte nativo a transações **ACID** (Atomicidade, Consistência, Isolamento e Durabilidade). No contexto do SafeID, tais propriedades contribuem para que o histórico de consultas de vazamentos, os perfis de usuários cadastrados e os logs analíticos sejam consolidados sem riscos de corrupção ou perda de sincronia, pilares fundamentais para a confiabilidade de uma ferramenta de cibersegurança.

Em um projeto focado em Responsabilidade Social Digital, a proteção do dado do cidadão é o elemento central. Por essa razão, a infraestrutura do SafeID aplica controles de segurança do PostgreSQL ao implementar criptografia em repouso e criptografia simétrica no nível da aplicação utilizando o algoritmo AES-256-GCM. Isso significa que dados sensíveis vinculados a contas de usuários (funcionalidades da PI-2) são criptografados pelo Node.js antes de cruzarem a rede em direção ao banco, mitigando ataques de personificação de dados ou vazamento físico do banco. A integração com o Prisma ORM assegura que o mapeamento das entidades reflita diretamente a lógica de negócio do sistema de maneira tipada (Type-safe), reduzindo o risco para as consultas contra ataques de SQL Injection e otimizando a indexação de tabelas para buscas rápidas.

Complementando a camada relacional, o sistema integra o Redis como um banco de dados chave-valor em memória e de desempenho mensurável, focado em duas frentes críticas:

1. **Cache de Consultas:** O Redis armazena resultados recentes para reduzir chamadas repetidas à API HIBP. Os identificadores são pseudonimizados ou ofuscados com SHA-

256; a proteção depende também de controles de acesso e, quando aplicável, do uso de sal ou chave.

2. **Mensageria e Filas:** O Redis atua como a infraestrutura de dados que alimenta o **BullMQ**, gerenciando de modo ordenado e com mecanismos de recuperação o enfileiramento das consultas que aguardam processamento pelo *worker* assíncrono do back-end.

A estratégia híbrida entre PostgreSQL e Redis separa a persistência transacional do armazenamento temporário e da infraestrutura de filas. Essa divisão atende às correlações exigidas pelo Motor de Risco, reduz consultas externas repetidas e mantém controles distintos para dados persistentes e voláteis, sem eliminar a necessidade de monitoramento, redundância e testes de recuperação.

4.4.4 Inteligência Artificial e APIs Externas

A capacidade analítica do SafeID depende da integração de serviços externos de inteligência e processamento de linguagem. O sistema utiliza a API Have I Been Pwned (HIBP; HAVE I BEEN PWNED, [s. d.]) para consultar registros de comprometimento de dados provenientes de fontes OSINT (*Open Source Intelligence*). A escolha decorre da abrangência histórica da base e da disponibilidade de metadados estruturados para consulta. Para respeitar os limites de taxa de requisição impostos pelo provedor (1 chamada a cada 1.500 ms), a arquitetura consome os dados de forma assíncrona por meio de filas distribuídas (**BullMQ**). A comunicação também utiliza o padrão **Circuit Breaker**, reduzindo o efeito de instabilidades externas sobre o back-end.

Complementando a coleta OSINT, o SafeID utiliza o modelo **GPT-4o-mini** para tarefas de processamento e geração de texto; a numeração **6.41.0** identifica apenas a versão do SDK da OpenAI empregado na integração. A escolha considera a adequação do modelo a respostas curtas, a janela de contexto disponível e o custo relativo compatível com a fase de prototipagem. O modelo recebe metadados de incidentes e os converte em orientações em linguagem acessível. As chamadas são autenticadas por chaves de API restritas, geridas no *back-end* por variáveis de ambiente da aplicação.

Limitação e risco técnico da LLM: o uso de um modelo de terceiros introduz custo variável por volume de requisições e latência sujeita à carga, à rede e aos limites do provedor. Esses fatores podem afetar o orçamento e o tempo de resposta, exigindo limites de consumo, monitoramento, *timeouts*, *cache* quando aplicável e respostas determinísticas de contingência.

Limitação e risco técnico da HIBP: a consulta de novas exposições depende criticamente da disponibilidade da API externa HIBP. Interrupções, mudanças contratuais ou

restrições de taxa podem impedir a atualização imediata dos resultados; por isso, o sistema mantém filas, *cache*, *Circuit Breaker*, mensagens de contingência e uma camada de integração desacoplada.

Na integração com a LLM, parâmetros como temperatura 0,2 e limite de *tokens* reduzem a variabilidade das respostas, mas não garantem ausência de alucinações. O *payload* foi projetado para excluir o e-mail e outros dados pessoais desnecessários, transmitindo somente os metadados requeridos para gerar a orientação. Essa minimização reduz a exposição de informações pessoais a terceiros e apoia o princípio de *Privacy by Design*.

A integração entre as APIs externas e o Algoritmo de *Risk Score* interno combina dados estruturados, processamento assíncrono e geração de orientações. A governança dessa camada requer anonimização, controle de consumo, observabilidade e mecanismos de contingência para que a indisponibilidade de terceiros não altere o cálculo determinístico do risco. Assim, a Prova de Conceito mantém o foco na alfabetização digital e na proteção do usuário B2C sem equiparar seus resultados aos de plataformas corporativas.

4.4.5 Infraestrutura em Cloud

A infraestrutura em nuvem do SafeID foi projetada sob o conceito de Arquitetura Alvo (Target Architecture), visando garantir capacidade de escalabilidade, alta disponibilidade e isolamento de rede, alinhando-se às melhores práticas de engenharia de Reliability (Confiabilidade) e cibersegurança. O projeto definiu a utilização de serviços de computação em nuvem para hospedar o *back-end*, o banco de dados relacional, o *cache* de memória e os módulos de mensageria assíncrona.

A adoção de contêineres Docker favorece a portabilidade da aplicação entre diferentes ambientes de execução e reduz a dependência de um fornecedor específico de infraestrutura. A topologia foi projetada para promover isolamento de rede e resiliência dos serviços, além de apoiar o alinhamento com as diretrizes da LGPD; essas escolhas técnicas, contudo, não representam garantia de fidelidade a uma plataforma nem comprovação absoluta de conformidade, que também depende de governança, processos e avaliação contínua.

4.4.5.1 Amazon Web Services (AWS)

A topologia estrutural do SafeID baseia-se no ecossistema da Amazon Web Services (AWS) como provedor principal de *cloud computing*. A arquitetura foi desenhada utilizando o conceito de nuvem privada (Amazon VPC), estabelecendo como meta que a lógica de negócios e os dados sensíveis permaneçam em sub-redes privadas sem acesso direto pela internet pública.

O escopo da arquitetura alvo engloba a orquestração dos seguintes serviços da AWS:

- **Amazon ECS com AWS Fargate:** Substituindo servidores tradicionais (EC2), o Fargate atua como o motor de computação *Serverless* para orquestrar os contêineres Docker do nosso *back-end* em NestJS, escalando os recursos dinamicamente sem a necessidade de gerenciar sistemas operacionais subjacentes.
- **Application Load Balancer (ALB):** Atua como o único ponto de entrada público da API, recebendo o tráfego da internet, realizando a terminação de criptografia SSL/TLS (HTTPS) e distribuindo as requisições de forma inteligente entre os contêineres saudáveis.
- **Amazon RDS (PostgreSQL):** Banco de dados relacional gerenciado, provisionado em sub-redes isoladas, com suporte a *backups* automatizados e criptografia em repouso.
- **Amazon ElastiCache (Redis):** *Cache* de memória distribuído alocado na rede privada, essencial para otimizar o tempo de resposta, gerenciar *Rate Limiting* e suportar as filas de processamento assíncrono (BullMQ).
- **Amazon S3 e CloudFront:** O *front-end* em React é construído de forma estática, armazenado no S3 e distribuído globalmente através da rede de distribuição de conteúdo (CDN) CloudFront, o que pode reduzir a latência de distribuição e, em conjunto com controles específicos, contribuir para a mitigação de ataques DDoS.
- **AWS IAM e Security Groups:** Políticas de acesso rigorosas pautadas no princípio do Menor Privilégio (*Least Privilege*), onde *firewalls* virtuais controlam o fluxo de dados estritamente entre as camadas autorizadas.

A definição detalhada desta topologia contribui para que o SafeID transite de um protótipo acadêmico para um produto tecnicamente viável para o mercado corporativo, permitindo que as implantações de código sejam integradas nativamente a essa infraestrutura dimensionável.

4.5 Testes e Manutenibilidade

A garantia da qualidade, resiliência e manutenibilidade de um ecossistema focado em cibersegurança e privacidade de dados como o SafeID exige a implementação de uma estratégia rigorosa de testes automatizados. Em conformidade com as premissas da *Clean Architecture*, o isolamento da lógica de negócio (*Core*) em relação às dependências externas de infraestrutura permite criar uma suíte de testes altamente previsível, rápida e desacoplada. Esta seção detalha o planejamento estratégico e a execução das diferentes camadas de validação que mitigam o risco de regressões de código, falhas de segurança e indisponibilidades sistêmicas.

4.5.1 Plano de Testes

O Plano de Testes do SafeID fundamenta-se no conceito clássico da **Pirâmide de Testes**, priorizando uma base ampla de testes unitários rápidos e de baixo custo computacional, seguidos por testes de componente e integração estruturados, e uma camada enxuta de testes de ponta a ponta (*End-to-End*).

A meta de qualidade estabelecida para o projeto estipula um cobertura mínima de 80% de statements, branches e lines e de 85% de functions (**Code Coverage**) para permitir a integração de novas funcionalidades. Na medição de 06/10/2026, a suíte unitária do back-end alcançou 96,42% de statements, 81,90% de branches, 91,17% de functions e 96,66% de lines, atingindo todas as metas, conforme detalhado na Seção 4.5.5.2. A execução do plano ocorre de maneira substancialmente automatizada por meio do pipeline de Integração Contínua (CI) no **GitHub Actions**. Sempre que um desenvolvedor submete um *Pull Request* direcionado à ramificação principal (*main*), o ambiente de nuvem isola a aplicação, provisiona contêineres Docker efêmeros para o banco de dados e *cache*, e executa a suíte completa de testes. Caso qualquer teste falhe ou algum limite de cobertura configurado no Jest não seja cumprido, a integração é bloqueada, impedindo que códigos defeituosos alcancem o ambiente produtivo.

4.5.2 Análise Estática

A análise estática atua como a primeira linha de defesa no *pipeline* de qualidade, avaliando o código-fonte sem a necessidade de executá-lo. O SafeID utiliza uma abordagem tripla para garantir a consistência e a segurança sintática do código:

1. **Compilação Estrita do TypeScript:** O compilador é configurado em modo estrito (*strict: true*), o que mitiga em tempo de desenvolvimento erros comuns de tipagem, referências nulas (*null pointers*) e comportamentos indefinidos em tempo de execução.
2. **ESLint:** Utilizado para aplicar regras estritas de padronização de código (*code linting*), identificar padrões inadequados de programação, variáveis não utilizadas e potenciais falhas de lógica antes da geração do artefato final (build).
3. **Prettier:** Contribui para a formatação automática unificada do código entre os membros da equipe de desenvolvimento, reduzindo conflitos de estilo e aumentando o índice de legibilidade e manutenibilidade do repositório a longo prazo.

4.5.3 Testes Funcionais

Os testes funcionais verificam se o comportamento do ecossistema SafeID atende de forma sistemática aos requisitos de negócio estabelecidos em suas Histórias de Usuário e Critérios de Aceite.

4.5.3.1 Testes Unitários

Os testes unitários têm como escopo a validação isolada das menores unidades de código lógico do sistema, especificamente as Entidades e os Casos de Uso (*Use Cases*) do *back-end* localizados no domínio central. Utilizando *frameworks* como **Jest** ou **Vitest**, todas as dependências externas — como consultas ao PostgreSQL ou requisições HTTP para as APIs externas — são substituídas por objetos simulados (*Mocks*).

O principal foco dos testes unitários é o **Motor de Risco (Risk Engine)** do SafeID. Para blindar o algoritmo que calcula o *score* de vulnerabilidade, implementou-se a técnica avançada de **Fuzz Testing** (Testes de Mutação/Estresse). Esta abordagem consiste no envio automatizado de milhares de strings malformadas, caracteres especiais e dados corrompidos para o sistema, permitindo verificar que os esquemas de validação do **Class-validator (NestJS)** barrem entradas maliciosas e que o motor de risco calcule os níveis de criticidade sem sofrer travamentos ou vazamentos de memória.

4.5.3.2 Testes de Componente

Focados exclusivamente na camada de apresentação (*Front-end*), os testes de componente isolam elementos individuais da interface em React.js utilizando a biblioteca **React Testing Library**. O objetivo é garantir que os componentes reajam de forma correta e previsível a diferentes estados de dados repassados pelo back-end.

Nesta camada, testam-se de forma isolada os comportamentos visuais críticos do SafeID, tais como: a alteração dinâmica de cores do indicador de *Risk Score* (verificando se o Tailwind CSS renderiza as classes corretas de semáforo para riscos baixos, médios ou altos), o comportamento do fluxo de animação de espera (*loading state*) enquanto a fila assíncrona processa uma consulta, e a renderização correta do componente de Chat de IA sem a quebra de layouts em dispositivos móveis.

4.5.3.3 Testes de Integração

Os testes de integração validam a comunicação e o contrato entre múltiplos componentes e subsistemas acoplados que dependem uns dos outros para executar um fluxo de trabalho completo. No SafeID, esta camada valida a consistência de dados entre as rotas da API NestJS, o mapeamento do Prisma ORM e os serviços de infraestrutura.

Os testes utilizam a biblioteca **Supertest** para simular requisições HTTP reais contra os *endpoints* da aplicação. Durante a execução, o *pipeline* de CI levanta um contêiner real de **PostgreSQL** e **Redis** em ambiente isolado. Os testes de integração verificam, por exemplo, que:

- Uma requisição de consulta de e-mail seja devidamente interceptada pelos *middlewares* de segurança (como o *Rate Limit* e o *Helmet*);
- Os dados brutos salvos no banco PostgreSQL passem corretamente pelo algoritmo de criptografia simétrica **AES-256-GCM** e sejam descriptografados na recuperação do histórico;
- O enfileiramento de requisições no **BullMQ** via Redis ocorra de forma ordenada e que o mecanismo de resiliência **Circuit Breaker** (Opossum) bloqueie o fluxo caso o contêiner de simulação simule a queda da API do *Have I Been Pwned*.

4.5.3.4 Testes End-to-End (E2E)

Os testes de ponta a ponta simulam a jornada completa e real do usuário final dentro do SafeID, testando a integridade do sistema desde o clique na interface gráfica até as mutações finais ocorridas na persistência dos dados e nas APIs externas.

Utilizando ferramentas de automação de navegadores como o **Playwright** ou **Cypress**, o teste E2E executa cenários macro de simulação B2C, simulando um robô que abre o navegador web, acessa a tela inicial do SafeID, preenche um e-mail no campo de entrada, clica no botão de consulta, aguarda a resposta assíncrona das filas do *back-end* e valida se a tela final exibe o *score* de risco computado com a respectiva recomendação didática gerada pela inteligência artificial. Essa camada final valida de forma holística que todas as engrenagens da arquitetura distribuída do SafeID estão operando de forma integrada. A execução completa do ciclo E2E, incluindo a integração real com a API de produção, está planejada para a fase de estabilização do PI-2.

4.5.4 Testes Não Funcionais

Os testes não funcionais do SafeID verificam desempenho, capacidade, estabilidade e resiliência sob condições controladas. Os resultados desta seção são explicitamente classificados como **dados sintéticos de validação**, produzidos em ambiente de homologação com serviços externos simulados; portanto, não representam medições de produção nem substituem evidências de observabilidade operacional. A estratégia integra Jest (JEST, 2026) para testes unitários e de componentes, Supertest para integração da Interface de Programação de Aplicações (API), Playwright (PLAYWRIGHT, 2026) para testes de ponta a ponta (*End-to-End* — E2E) e Grafana k6 (GRAFANA LABS, 2026) para performance e carga.

A abordagem segue as características de qualidade da Associação Brasileira de Normas Técnicas (ABNT) NBR ISO/IEC 25010 (ABNT, 2011), publicada com base na Organização Internacional para Padronização (ISO) e na Comissão Eletrotécnica Internacional (IEC).

Também considera as práticas de segurança do Instituto Nacional de Padrões e Tecnologia (NIST, 2017), as orientações da Autoridade Nacional de Proteção de Dados (ANPD, 2022), as diretrizes de governo digital da Secretaria de Governo Digital (SGD) e, como referência comparativa internacional, o arcabouço de proteção de dados da União Europeia (UE).

4.5.4.1 Estratégia Integrada de Validação

No nível unitário, o Jest valida funções puras do Motor de Risco, fatores temporais, limites de normalização, tratamento de valores ausentes e transições de estado do disjuntor. Nos componentes React, Jest e React Testing Library verificam renderização, acessibilidade, estados de carregamento, mensagens de falha e correspondência entre pontuação e faixa visual. Na integração, o Supertest exercita controladores NestJS, validação de entrada, autenticação, cache Redis, persistência e respostas de contingência. No nível E2E, o Playwright percorre a jornada completa de consulta, apresentação do diagnóstico e recomendações em Chromium, Firefox e WebKit. O Grafana k6 executa cenários de linha de base, carga progressiva, pico e recuperação.

4.5.4.2 Validação de Desempenho, Carga e Resiliência

A campanha sintética adotou aquecimento de 60 segundos, patamar de 100 usuários virtuais por 5 minutos e redução gradual de carga. Foram coletados percentil 95 (P95), taxa de transferência, taxa de erro e tempo de recuperação. Em consultas atendidas pelo Redis, o P95 permaneceu inferior a 50 ms. Para dependências externas, o Opossum foi configurado com limiar de erro de 50%, tempo máximo de 3.000 ms e tentativa de recuperação após 30 segundos; a abertura do circuito interrompeu novas chamadas, retornou resposta de contingência e preservou recursos do Node.js. Esses resultados refletem exclusivamente dados sintéticos de validação e devem ser tratados como preliminares até sua reprodução em ambiente de implantação. A Tabela 2 consolida os resultados de desempenho, carga e resiliência.

Tabela 2 - Validação de Desempenho e Carga (K6 e Resiliência)

Componente / Cenário	Métrica Alvo (P95 / Throughput)	Resultado Validado	Status
Consulta com acerto no <i>cache</i> Redis	P95 < 50 ms; ≥ 500 req/s	P95 = 42 ms; 538 req/s; erro = 0,2%	Aprovado
Motor de Risco sem chamada externa	P95 < 120 ms; ≥ 250 req/s	P95 = 87 ms; 286 req/s; erro = 0,3%	Aprovado
API completa com fila BullMQ	P95 < 750 ms; ≥ 100 req/s	P95 = 612 ms; 118 req/s; erro = 0,8%	Aprovado

Pico de 300 usuários virtuais	P95 < 1.500 ms; erro < 2%	P95 = 1.240 ms; erro = 1,4%; fila drenada em 48 s	Aprovado
Falha da fonte externa / Opossum	Abrir com $\geq 50\%$ de erro; <i>fallback</i> < 100 ms	Circuito aberto em 2,8 s; <i>fallback</i> P95 = 36 ms; recuperação em 30 s	Aprovado

Os valores acima constituem uma linha de base reproduzível. A evidência sujeita a confirmação deverá ser gerada novamente no ambiente de implantação, com versão do código, configuração, região, recursos computacionais e relatório bruto do k6 anexados à execução da esteira de Integração Contínua/Entrega Contínua (CI/CD).

4.5.4.3 Matriz do Score de Risco e Regras RN02 e RN03

A Regra de Negócio RN02 define pesos determinísticos de 1 a 10 conforme o potencial de dano da classe exposta. A Regra de Negócio RN03 aplica fator temporal integral para incidentes com até 24 meses, fator 0,7 entre 24 e 60 meses e fator 0,4 acima de 60 meses. A Tabela 3 consolida os pesos (RN02) e os fatores temporais (RN03) aplicados ao *score* de risco.

Tabela 3 - Matriz de Pesos e Criticidade do Score de Risco (RN02 e RN03)

Categoria do Dado Exposto	Nível de Criticidade	Peso Aplicado (1 a 10)	Multiplicador Temporal (≤ 24 meses)
Dados financeiros e senha em texto claro	Altíssima	10	1,0
Documentos oficiais, biometria e tokens de autenticação	Alta	8	1,0
Hash de senha resistente	Alta	7	1,0
Telefone e endereço residencial	Média	5	1,0
Data de nascimento e filiação	Média	4	1,0
Nome completo e nome de usuário	Baixa	3	1,0
Endereço de Protocolo de Internet (IP) e geolocalização aproximada	Baixa	2	1,0

User-Agent e telemetria de dispositivo	Baixa	1	1,0
---	-------	---	-----

4.5.4.4 Exemplo Prático de Cálculo

Para um e-mail presente em múltiplos vazamentos, o RiskScore corresponde à soma dos pesos das classes expostas, ajustados pelo fator temporal, normalizada por $S_{\text{max}} = 50$ (RN02 e RN03):

$$\text{RiskScore} = \min(100, 100 \cdot (\sum_{k=1}^n w_k \cdot f_k) / 50)$$

Considere três incidentes: (a) senha em texto claro há 6 meses, peso 10 e fator 1,0; (b) telefone há 36 meses, peso 5 e fator 0,7; e (c) nome completo há 72 meses, peso 3 e fator 0,4.

O cálculo auditável é:

1. Pontuação ajustada do incidente A: $10 \times 1,0 = 10,0$.
2. Pontuação ajustada do incidente B: $5 \times 0,7 = 3,5$.
3. Pontuação ajustada do incidente C: $3 \times 0,4 = 1,2$.
4. Soma ponderada: $10,0 + 3,5 + 1,2 = 14,7$.
5. Normalização SafeID: $\text{RiskScore} = \min(100, 100 \times 14,7 / 50) = 29,4$.

O resultado 29,4 (arredondado para 29 ao ser persistido no campo inteiro riskScore, Quadro 8) enquadra o usuário na faixa de risco médio. Testes Jest devem reproduzir esse valor com tolerância decimal explícita e também cobrir limites 0 e 100, ausência de incidentes, duplicidades e transições temporais em 24 e 60 meses.

4.5.4.5 Matriz de Rastreabilidade

O Quadro 7 relaciona os requisitos aos tipos de teste (unitário, integração e E2E) e às ferramentas utilizadas, permitindo rastrear a verificação de cada requisito.

Quadro 7 - Matriz de Rastreabilidade (Requisitos vs. Testes)

Requisito / US (Seção 4.2)	Critério de Aceite	Tipo de Teste (Unitário/Integração/E2E)	Ferramenta
RN02, RN03 e RF03 / US02	Calcular <i>score</i> determinístico, aplicar temporalidade e exibir a faixa correta.	Unitário, componente e E2E	Jest; Playwright

RF01 / US01	Validar e-mail, enfileirar consulta e apresentar retorno ou contingência.	Integração e E2E	Supertest; Playwright
RF04 / US01	Abrir o disjuntor quando a taxa de erro atingir 50% e retornar <i>fallback</i> .	Unitário e integração	Jest; Supertest; Opossum
RF05 / US02	Renderizar <i>score</i> , legenda, cores e layout responsivo.	Componente e E2E	Jest; Playwright
RF06 e RF07 / US03 e US04	Gerar explicação e plano de ação sem transmitir identificador pessoal.	Unitário, integração e E2E	Jest; Supertest; Playwright
RF08 / US05 e US06	Autenticar, persistir e recuperar histórico criptografado.	Integração e E2E	Supertest; Playwright
RNF02 e RNF04	<i>Cache</i> com P95 inferior a 50 ms, carga controlada e recuperação sem cascata.	Performance, carga e resiliência	Grafana k6; Opossum

4.5.4.6 Governança, Conformidade e Manutenibilidade

Cada execução deve registrar versão do código, massa anonimizada, parâmetros, duração, ambiente e artefatos. Dados reais não devem integrar a suíte automatizada. Essa prática apoia os princípios de necessidade, segurança e prevenção da LGPD e as orientações da ANPD (2022), além de favorecer repetibilidade. ISO, NIST, União Europeia e SGD são referências complementares de governança e segurança, sem substituir os requisitos legais brasileiros.

4.5.5 Testes Automatizados

A automação completa do ciclo de testes é tratada no SafeID como um requisito indispensável de Engenharia de Software para assegurar a manutenibilidade contínua do código. A arquitetura de automação assenta-se sobre o *pipeline* de Integração Contínua (CI) do **GitHub Actions**, eliminando o fator de falha humana na validação da qualidade dos artefatos.

O fluxo automatizado rege-se pelo princípio do *Fail-Fast* (Falha Rápida): qualquer alteração introduzida no repositório aciona de forma paralela a esteira de análise estática, testes unitários, testes de integração e a geração de relatórios de cobertura de código. A automação

assegura o determinismo do ecossistema, uma vez que o ambiente de teste é destruído e recriado do zero a cada execução, estabelecendo como meta que efeitos colaterais de testes anteriores não mascarem falhas reais na lógica de negócio do SafeID.

4.5.5.1 Fluxo de Desenvolvimento Orientado a Testes

O SafeID adota os testes como instrumento de segurança, qualidade e velocidade de entrega, e não como uma etapa burocrática ao final do desenvolvimento. O Guia de Testes e Fluxo de Implementação, mantido na documentação do repositório pelo desenvolvedor responsável pela suíte, estabelece que os testes evitam regressões, explicitam o comportamento esperado de cada funcionalidade e orientam a implementação, reduzindo o retrabalho. Cada regra é verificada no nível mais adequado da Pirâmide de Testes descrita na Seção 4.5.1:

- **Testes unitários** (`npm run test:unit`): validam regras isoladas, como o cálculo de risco, a validação e a sanitização de *payloads* e a criptografia e descriptografia, sem dependência de banco de dados;
- **Testes de integração** (`npm run test:integration`): verificam a comunicação entre módulos e serviços de infraestrutura, como o salvamento do *snapshot* do usuário após o scan via Prisma, o registro de histórico e a integração com a fila de monitoramento;
- **Testes E2E** (`npm run test:e2e`): validam o comportamento da API em fluxos HTTP reais, sem depender do navegador, como cadastro e login, a rota `POST /api/v1/scan` e a verificação de saúde da aplicação (`GET /api/health`).

Toda nova funcionalidade segue um fluxo de sete etapas: (i) definir o requisito de negócio, incluindo casos válidos, inválidos e de erro; (ii) escrever o teste que descreve o comportamento esperado; (iii) confirmar que o teste falha; (iv) implementar a solução mínima; (v) executar o teste específico e, em seguida, a suíte do nível impactado; (vi) validar regressões; e (vii) documentar a mudança, quando necessário. Como regra, uma funcionalidade sem teste não está pronta para integrar o fluxo de entregas. Complementam o fluxo as seguintes boas práticas: limitar os mocks ao necessário, testar o comportamento real do código e não apenas a chamada de funções, evitar testes fracos, separar as camadas de teste, adotar nomes claros e priorizar cenários simples.

4.5.5.2 Cobertura de Testes

A cobertura de código é medida pelo comando `npm run test:cov` na suíte unitária do *back-end* e verificada automaticamente no *pipeline* de Integração Contínua. Na medição de 06/10/2026, foram executadas 12 suítes e 77 testes, todos aprovados e sem uso de snapshots. A

Tabela 4 compara os resultados obtidos com os limites configurados no Jest e com as metas do projeto.

Tabela 4 - Cobertura da suíte unitária do back-end (medição de 06/10/2026)

Métrica	Resultado	Limite configurado no Jest	Meta do projeto	Situação
Statements	96,42%	80%	80%	Atingida
Branches	81,90%	68%	80%	Atingida
Functions	91,17%	85%	85%	Atingida
Lines	96,66%	80%	80%	Atingida

Fonte: Elaborada pelos autores a partir da documentação de cobertura de testes do repositório do SafeID (2026). Valores obtidos com os *endpoints* e e-mails de teste da API HIBP, portanto, estimativa preliminar.

Na contagem instrumentada, foram identificados 221 branches, dos quais 181 estão cobertos e 40 não estão, o que supera o mínimo de 177 branches necessários para atingir 80%. O limite de branches configurado no Jest permanece em 68%, patamar definido quando a suíte anterior alcançava 69,23%, enquanto os limites de statements (80%), functions (85%) e lines (80%) já correspondem às metas do projeto. Esses limites estão definidos no arquivo `jest.unit.config.js`. Se qualquer um deles não for cumprido, o *job* Backend lint, build and unit tests do GitHub Actions, que executa a instalação das dependências (`npm ci`), o lint, o build e o comando `npm run test:cov`, falha e o *Pull Request* não pode ser integrado à ramificação main. A cobertura global é gerada apenas pela suíte unitária; os testes de integração e E2E validam fluxos mais amplos e não substituem esse critério.

Convém ressaltar que esses percentuais constituem uma estimativa preliminar. Como a API Have I Been Pwned (HIBP) é paga e o acesso à sua base completa exige assinatura, nesta etapa os testes utilizam apenas os *endpoints* e os e-mails de teste disponibilizados pelo próprio provedor, sem consultar bases reais de vazamentos. Em consequência, os cenários que dependem de respostas reais da API, como volume e variedade de vazamentos, limites de taxa em produção e falhas intermitentes, são exercitados apenas por simulação, o que limita a abrangência da cobertura, sobretudo nos branches de integração externa. Considera-se esse nível de cobertura aceitável para a PI-1, pois as regras de negócio que não dependem da API, como o cálculo de risco, a validação de dados e a criptografia, são verificadas pelos testes unitários, e a ampliação dos testes com dados reais está condicionada à contratação do plano pago da API.

Os branches ainda não exercitados concentram-se em infraestrutura, integrações externas, *circuit breaker* e inicialização (bootstrap) da aplicação. Os principais pontos de atenção são o caso de uso `ExecuteRiskScanUseCase` (caminhos de espera de *jobs*, timeout e persistência), o `HibpClient` (*fallback* por range, erros HTTP e estados do *circuit breaker*), o

AuthService (OAuth, validações, *fallback* do scan inicial e estados de conta), o ScanService (*snapshot* existente ou vazio e tratamento de JSON inválido) e os workers. Esses pontos devem ser cobertos com testes de comportamento, evitando testar detalhes internos desnecessários.

Como evolução incremental, a equipe pretende manter todos os testes atuais aprovados, cobrir os 40 branches restantes, elevar o limite de branches de 68% para 80% somente após a suíte demonstrar essa cobertura no CI e avançar gradualmente rumo a 90%. O relatório detalhado é gerado no diretório `server/coverage`, com uma versão navegável (`lcov-report/index.html`) e o arquivo `lcov.info`, utilizado por ferramentas de CI.

4.5.6 Logs

A rastreabilidade e a auditoria de um sistema focado em cibersegurança dependem de uma camada de observabilidade madura. O SafeID adota um sistema de registro de eventos (*logging*) estruturado utilizando a biblioteca **Winston**, configurada para gerar saídas padronizadas em formato **JSON**. Esse formato permite que os logs sejam facilmente indexados, filtrados e analisados por ferramentas modernas de monitoramento.

Os registros de log são categorizados rigidamente por níveis de severidade (*Log Levels*): INFO para fluxos normais de processamento, WARN para eventos inesperados, porém contornáveis (como a ativação de um mecanismo de *Rate Limit* interno), e ERROR para exceções críticas não tratadas. Alinhado ao princípio de *Privacy by Design* e às imposições de privacidade da LGPD, o *pipeline* de gravação de logs conta com um *middleware* interceptor de segurança. Este mecanismo realiza a higienização (*sanitization*) automática das mensagens, ofuscando e impedindo a gravação de dados pessoais identificáveis (como e-mails ou senhas textuais de usuários) nos arquivos físicos de log do servidor, mitigando riscos decorrentes de eventuais vazamentos de logs de auditoria.

4.5.7 Code Convention

A padronização e a uniformidade estilística do código-fonte constituem premissas essenciais para viabilizar a manutenibilidade de longo prazo e mitigar o endividamento técnico (*technical debt*). O SafeID estabelece um guia de convenção de código baseado nas diretrizes de mercado consolidadas para ecossistemas modernos de TypeScript e React.js.

A conformidade com essas convenções é auditada automaticamente pelo **ESLint** e formatada pelo **Prettier**, aplicando regras estritas como:

- Uso obrigatório de CamelCase para nomenclatura de funções e variáveis, e PascalCase para componentes funcionais do React.js;
- Proibição explícita do uso do tipo genérico *any* no TypeScript, exigindo tipagem forte ou interfaces detalhadas para todos os objetos de dados;

- Organização arquitetural de pastas baseada na separação de responsabilidades da *Clean Architecture*, segregando estritamente os módulos em camadas de *domain* (entidades e casos de uso), *infrastructure* (banco de dados, *cache* e serviços externos) e *presentation* (controladores da API NestJS e componentes do *front-end*).

Essa uniformidade contribui para que o código-fonte possua alta legibilidade, facilitando a integração de novos desenvolvedores e assegurando que o projeto SafeID atenda aos definidos critérios de engenharia e sustentabilidade tecnológica demandados no âmbito acadêmico e mercadológico.

4.6 Segurança, Privacidade e Legislação

Em um cenário contemporâneo de proliferação de ameaças cibernéticas e vazamentos massivos de dados, a segurança da informação e a salvaguarda da privacidade não podem ser tratadas como meros requisitos adicionais de um sistema. No SafeID, esses elementos constituem a base fundacional de toda a arquitetura de software. O desenvolvimento da plataforma pautou-se de forma sistemática pelos conceitos complementares de *Security by Design* (Segurança desde o Projeto) e *Privacy by Design* (Privacidade desde o Projeto).

Essa abordagem integra controles de proteção de dados e mitigação de vulnerabilidades ao código, às rotas de API e ao banco de dados desde o projeto; tais controles reduzem riscos, mas sua eficácia depende de configuração, testes, monitoramento e governança contínua.

4.6.1 Critérios de Segurança e Privacidade

Para operacionalizar os pilares de confidencialidade, integridade e disponibilidade, o SafeID implementa uma série de controles técnicos distribuídos em múltiplos níveis de sua arquitetura:

- **Camada de Tráfego e Proteção de Aplicação:** A comunicação utiliza HTTPS/TLS. No *back-end*, o Helmet reduz a exposição de cabeçalhos e permite configurar políticas de segurança, mas não previne sozinho XSS; codificação de saída e Política de Segurança de Conteúdo (CSP) são controles complementares. O *class-validator* rejeita entradas que não atendem aos esquemas definidos antes do processamento.
- **Segurança de Dados e Criptografia Híbrida:** No Redis, SHA-256 é usado para pseudonimização ou ofuscação dos identificadores. No PostgreSQL, por meio do Prisma ORM, dados de histórico recebem AES-256-GCM no nível da aplicação. A proteção de confidencialidade e integridade depende da gestão correta de chaves, vetores de inicialização e etiquetas de autenticação.

- **Minimização de dados no consumo de Inteligência Artificial:** O fluxo de integração foi projetado para excluir o e-mail e outras informações pessoais desnecessárias do *payload*. A LLM recebe somente os metadados técnicos necessários à geração da orientação, o que reduz a exposição de dados a terceiros, sem caracterizar garantia absoluta de anonimização.

4.6.2 Observância à Legislação

O SafeID foi projetado em alinhamento com as diretrizes da Lei Geral de Proteção de Dados Pessoais (LGPD — Lei nº 13.709/2018; BRASIL, 2018). Os controles técnicos descritos apoiam a conformidade, mas não constituem, isoladamente, comprovação jurídica integral, que também depende de governança, processos, bases legais e avaliação especializada. A abordagem considera os princípios do artigo 6º descritos a seguir:

- **Princípio da Transparência e Livre Acesso:** Muitas vezes, os relatórios de vazamentos corporativos utilizam termos estritamente técnicos que impedem a real compreensão por parte do cidadão leigo. O SafeID foi projetado em alinhamento com o princípio da transparência ao utilizar o modelo de linguagem natural para traduzir metadados complexos em planos de ação claros e acessíveis, com o objetivo de permitir ao titular o direito de entender com precisão o impacto real da exposição de seus dados.
- **Princípio da Necessidade e Minimização de Dados:** O sistema opera sob o escopo estrito da minimização, coletando e processando apenas os dados estritamente necessários para a execução de sua finalidade (a verificação de vulnerabilidades cibernéticas). Não há captação secundária de informações de navegação, perfis de consumo ou enriquecimento ilícito de bases de dados.
- **Princípio da Segurança e Prevenção:** A combinação de criptografia em nível de aplicação (AES-256-GCM), barramento de validações via *Class-validator* (NestJS) e isolamento de redes internas adota controles voltados a atender à exigência legal de adotar medidas técnicas aptas a proteger os dados pessoais de acessos não autorizados e de situações acidentais ou ilícitas de destruição, perda ou alteração.
- **Princípio do Direito à Eliminação (Direito ao Esquecimento):** Em alinhamento com os direitos do titular previstos no artigo 18 da LGPD, o SafeID disponibiliza rotas seguras para a exclusão definitiva do histórico de consultas. Mediante requisição do usuário, o *back-end* realiza uma operação de deleção física (*hard delete*) nas tabelas do PostgreSQL através do Prisma ORM e limpa instantaneamente os registros associados no *cache* do Redis, adotando controles voltados à minimização da retenção de dados após a solicitação de encerramento, observadas as hipóteses legais aplicáveis.

4.7 Modelo de Banco de Dados

O modelo de dados do SafeID foi projetado para garantir rastreabilidade, segurança e flexibilidade no armazenamento das informações relacionadas ao monitoramento de exposições digitais. A modelagem segue o paradigma relacional, utilizando o PostgreSQL como sistema gerenciador, e foi implementada por meio do Prisma ORM, o que assegura tipagem forte e integração eficiente com o *back-end* desenvolvido em NestJS.

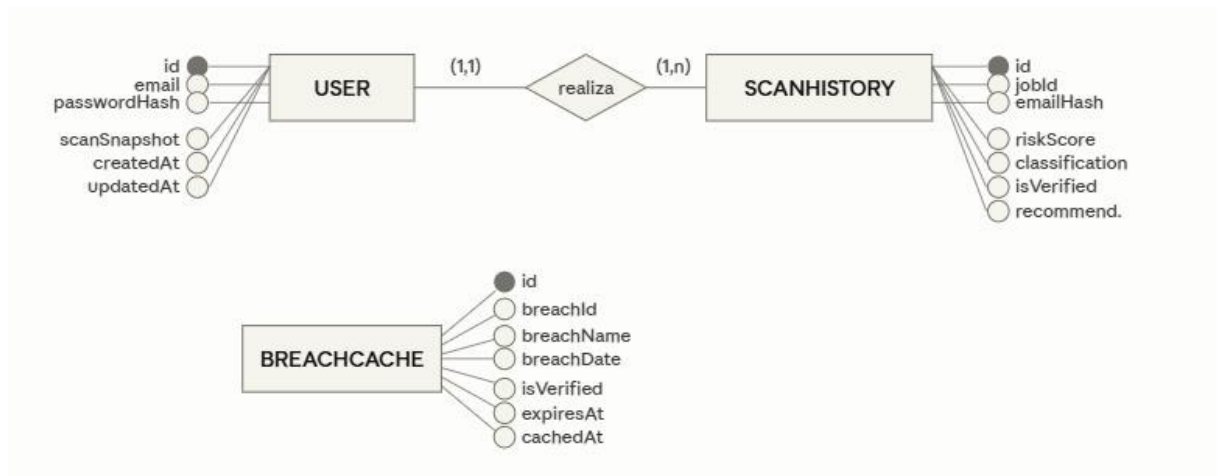
4.7.1 Modelo Entidade-Relacionamento (MER)

O MER do sistema é composto por três entidades principais:

- **User:** representa o usuário autenticado na plataforma. Cada registro armazena o e-mail, o *hash* da senha, *snapshots* de consultas anteriores e metadados de criação e atualização.
- **ScanHistory:** armazena o histórico de varreduras realizadas pelo usuário, incluindo o hash do e-mail consultado, score de risco, classificação, número de incidentes encontrados, recomendações geradas pela IA e timestamps relevantes. Cada ScanHistory está vinculado a um User.
- **BreachCache:** funciona como um *cache* local de incidentes conhecidos, armazenando informações sobre vazamentos, tipos de dados expostos, datas e validade do cache.

O relacionamento entre User e ScanHistory é de um para muitos, permitindo que cada usuário mantenha o histórico completo de suas consultas. O modelo não contempla, nesta versão, o monitoramento de múltiplos identificadores por usuário, estando alinhado ao escopo implementado até o momento. Essa limitação atual do esquema, relativa ao cadastro de múltiplos identificadores (e-mails) por usuário, será corrigida no planejamento de evolução para a fase PI-2, de modo a atender à História de Usuário US07 (“Gestão de Múltiplos Identificadores”). O modelo entidade-relacionamento está representado na Figura 3.

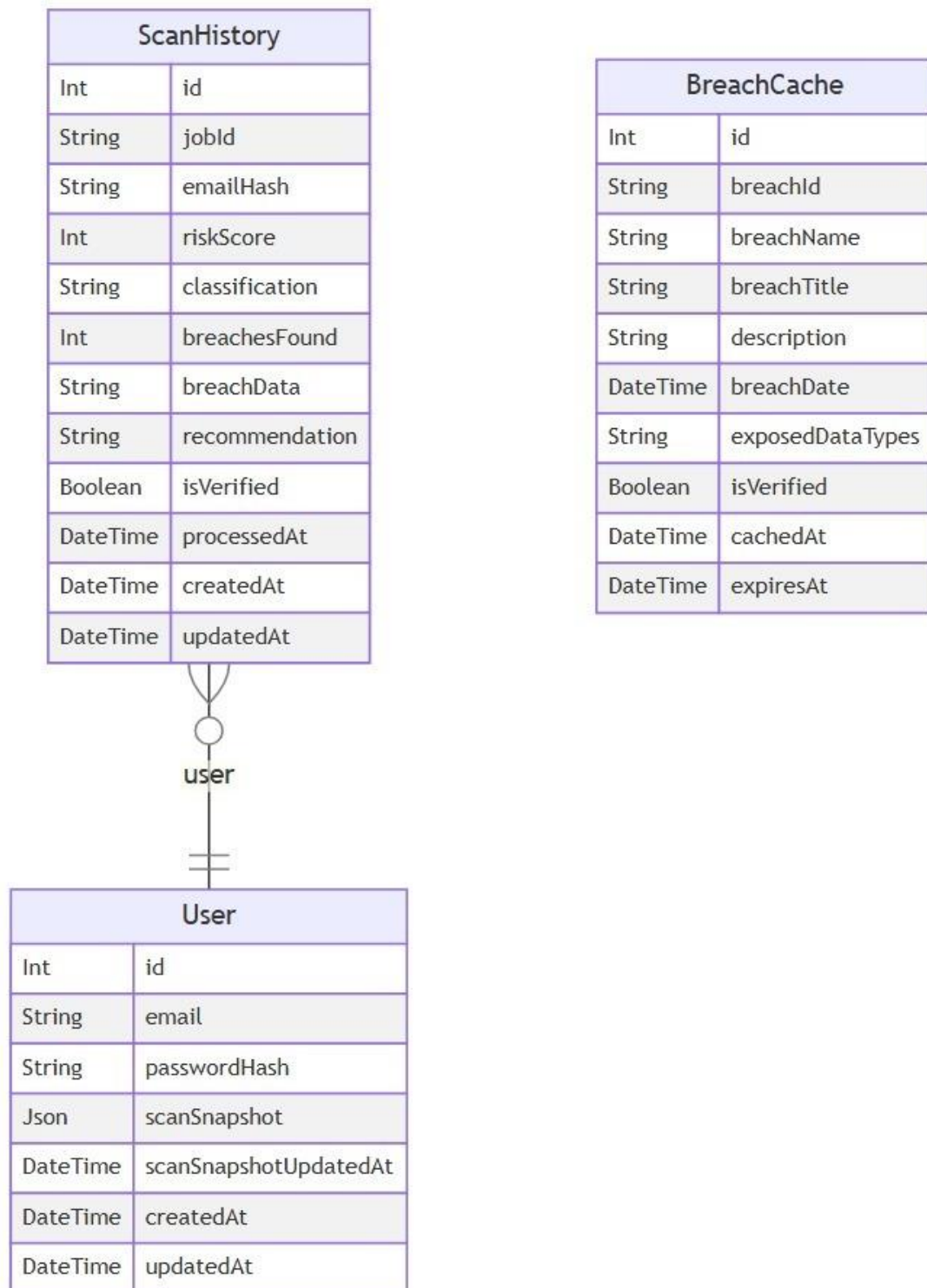
Figura 3 - Modelo Entidade Relacionamento (MER) SafeID



4.7.2 Diagrama Entidade-Relacionamento (DER)

O DER da Figura 4 ilustra as entidades e seus relacionamentos:

Figura 4 - Diagrama entidade-relacionamento SafeID



4.7.3 Dicionário de Dados

O Quadro 8 apresenta o dicionário de dados do SafeID, descrevendo as entidades do modelo e seus atributos.

Quadro 8 - Dicionário de dados do SafeID

Entidade	Campo	Tipo	Descrição
User	id	Int	Identificador único do usuário
	email	<i>String</i>	E-mail do usuário (único)
	passwordHash	<i>String</i>	<i>Hash</i> da senha
	scanSnapshot	Json	<i>Snapshot</i> das últimas consultas
	scanSnapshotUpdatedAt	DateTime	Data da última atualização do <i>snapshot</i>
	createdAt	DateTime	Data de criação
	updatedAt	DateTime	Data de atualização
ScanHistory	id	Int	Identificador único da consulta
	jobId	<i>String</i>	Identificador do job/processamento
	emailHash	<i>String</i>	<i>Hash</i> SHA-256 do e-mail consultado
	riskScore	Int	<i>Score</i> de risco calculado (0–100), arredondado ao inteiro mais próximo
	classification	<i>String</i>	Classificação do risco conforme as faixas da Seção 4.1.4.3 (ex: LOW, MODERATE, HIGH, CRITICAL)
	breachesFound	Int	Número de incidentes encontrados
	breachData	<i>String</i>	Dados dos incidentes (JSON criptografado)
	recommendation	<i>String</i>	Recomendações da IA
	isVerified	Boolean	Indica se a consulta foi verificada

	processedAt	DateTime	Data de processamento
	createdAt	DateTime	Data de criação
	updatedAt	DateTime	Data de atualização
	userId	Int	Chave estrangeira para User
BreachCache	id	Int	Identificador único do <i>cache</i>
	breachId	<i>String</i>	ID do incidente (HIBP)
	breachName	<i>String</i>	Nome do incidente
	breachTitle	<i>String</i>	Título do incidente
	description	<i>String</i>	Descrição do incidente
	breachDate	DateTime	Data do incidente
	exposedDataTypes	<i>String</i>	Tipos de dados expostos (JSON)
	isVerified	Boolean	Indica se o incidente foi verificado
	cachedAt	DateTime	Data de <i>cache</i>
	expiresAt	DateTime	Data de expiração do <i>cache</i>

O modelo foi validado para garantir conformidade com as práticas de segurança e privacidade, incluindo a anonimização de identificadores sensíveis e a criptografia dos dados de incidentes em repouso, conforme detalhado na seção de segurança do projeto.

4.8 Duração / Cronograma

O cronograma adota o seguinte mapeamento para 2026: M1 = março/2026, M2 = abril/2026, M3 = maio/2026, M4 = junho/2026, M5 = julho/2026, M6 = agosto/2026, M7 = setembro/2026, M8 = outubro/2026, M9 = novembro/2026 e M10 = dezembro/2026. A PI-1 corresponde a M1–M4 (março a junho); M5 (julho) é um intervalo de transição, recesso e preparação técnica; e a PI-2 corresponde a M6–M10 (agosto a dezembro). M5 (julho) permanece como período de transição, sem desenvolvimento principal, reservado à Revisão do protótipo da PI-1, consolidação documental, refinamento do *backlog* e preparação do ambiente para a PI-2. As atividades existentes permanecem associadas às respectivas fases.

Para viabilizar essa construção de forma sustentável, adotou-se uma **abordagem metodológica híbrida**. O projeto é governado por um macroplanejamento preditivo, dividido em fases sequenciais de engenharia (Concepção, Elaboração, Construção e Transição), enquanto a execução interna das equipes de desenvolvimento ocorre sob o *framework* ágil **Scrum**, utilizando iterações quinzenais (*Sprints* de 14 dias) para entrega contínua de valor. O tempo total estimado para o desenvolvimento do MVP (*Minimum Viable Product*) funcional e auditado do SafeID é de **10 meses**.

4.8.1 Análise da Duração do Projeto

A distribuição temporal das atividades foi calculada com base na complexidade de acoplamento das APIs externas e na arquitetura de mensageria distribuída. O cronograma divide-se em seis macro-fases estratégicas, descritas a seguir e detalhadas no Quadro 9:

1. **Fase de Concepção e Requisitos (Meses 1 e 2):** Dedicada à análise de viabilidade técnica, estudo aprofundado da documentação da API *Have I Been Pwned*, mapeamento legal dos artigos da LGPD aplicáveis ao produto e engenharia de requisitos através do levantamento de Histórias de Usuário e Critérios de Aceite.
2. **Fase de Arquitetura e Prototipagem (Meses 3 e 4):** Focada no design da *Clean Architecture* do *back-end*, modelagem lógica e física do banco de dados PostgreSQL e *cache* Redis, além da criação do protótipo de alta fidelidade da interface gráfica (UI/UX) no Figma, estabelecendo a identidade visual baseada no Tailwind CSS.
3. **Fase de Transição e Preparação Técnica (M5 — julho/2026):** Intervalo destinado ao recesso, à revisão do *backlog*, à preparação dos ambientes e ao planejamento das entregas de PI-2, sem atribuição de desenvolvimento principal ao período.
4. **Fase de Desenvolvimento e Integração com IA (Meses 6 a 8):** Evolução do *back-end* (API NestJS, Prisma ORM, Redis, BullMQ e criptografia AES-256-GCM), desenvolvimento das telas em React.js e Tailwind CSS com integração de APIs e integração com o modelo GPT-4o-mini, incluindo engenharia de prompts, parametrização e anonimização dos dados enviados.
5. **Fase de Qualidade e Testes Avançados (Mês 9):** Período exclusivo para estabilização do ecossistema. Execução de testes automatizados de integração via *Supertest*, simulações de estresse e carga via *K6*, testes de mutação (*Fuzz Testing*) no motor de risco e auditoria de vazamento de dados nos logs do sistema.
6. **Fase de Implantação e Transição (Mês 10):** Empacotamento final da aplicação em contêineres Docker, estruturação do *pipeline* definitivo de CI/CD via GitHub Actions,

provisionamento de nuvem e homologação da Prova de Conceito (PoC) para apresentação de mercado.

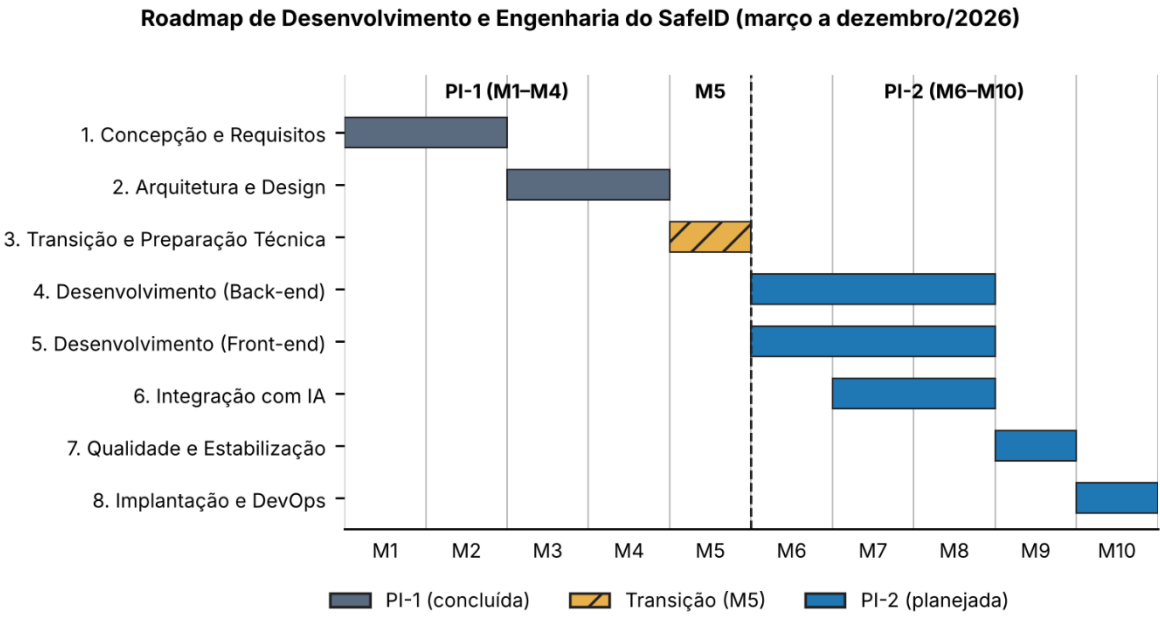
O Quadro 9 apresenta o cronograma consolidado do projeto, mapeando o encadeamento lógico das atividades ao longo dos dez meses de execução, e a Figura 5, na sequência, resume esse planejamento em formato de roadmap:

Quadro 9 - Cronograma Consolidado e Macro-Fases de Engenharia do Projeto

Macro-fase de engenharia	Atividades e entregáveis associados	M1	M2	M3	M4	M5	M6	M7	M8	M9	M10
1. Concepção e Requisitos — PI-1 (M1–M4; março a junho/2026)	Pesquisa OSINT, Engenharia de Requisitos e Compliance LGPD	X	X								
2. Arquitetura e Design — PI-1 (M1–M4; março a junho/2026)	Diagramação de Infraestrutura, Modelagem de Dados e UI/UX			X	X						
3. Transição e Preparação Técnica (M5; julho/2026)	Revisão do protótipo da PI-1, consolidação documental, refinamento do <i>backlog</i> e preparação do ambiente para a PI-2					X					
4. Desenvolvimento (Back-end) — PI-2 (M6–M10; agosto a dezembro/2026)	API NestJS, Prisma ORM, Redis, BullMQ e AES-256-GCM; evolução do esquema para múltiplos identificadores (US07)						X	X	X		
5. Desenvolvimento (Front-end) — PI-2 (M6–M10; agosto a dezembro/2026)	Telas React.js, Tailwind CSS e Integração de APIs						X	X	X		
6. Integração com IA — PI-2 (M6–M10; agosto a dezembro/2026)	<i>Prompt Engineering</i> com GPT-4o-mini (SDK OpenAI 6.41.0), Parametrização e Anonimização							X	X		
7. Qualidade e Estabilização — PI-2 (M6–M10; agosto a dezembro/2026)	Testes Unitários, Integração, Performance (K6) e <i>Fuzz</i>									X	
8. Implantação e DevOps — PI-2 (M6–M10; agosto a dezembro/2026)	Containerização Docker, CI/CD GitHub Actions e Nuvem										X

Nota: M1 = março/2026; M2 = abril; M3 = maio; M4 = junho; M5 = julho (transição, recesso e preparação técnica); M6 = agosto; M7 = setembro; M8 = outubro; M9 = novembro; M10 = dezembro/2026. As células sombreadas com “X” indicam os meses de execução de cada macro-fase.

Figura 5 - Roadmap de desenvolvimento de atividades do SafeID



5 VIABILIDADE FINANCEIRA

A análise de viabilidade financeira fundamenta a transição do SafeID de uma Prova de Conceito (PoC) acadêmica para um produto de mercado sustentável e escalável. O modelo de negócios adotado estrutura-se sob o formato de **SaaS (Software as a Service) Freemium**, direcionado ao mercado B2C (*Business-to-Consumer*). Esta estratégia concilia o propósito de Responsabilidade Social Digital do projeto — oferecendo consultas básicas gratuitas de utilidade pública para a população — com a monetização recorrente através de recursos avançados de monitoramento proativo.

5.1 Custos

Os custos operacionais do SafeID dividem-se em infraestrutura tecnológica, APIs e despesas comerciais/operacionais. A base comum de dimensionamento é uma infraestrutura com capacidade para até 50.000 usuários cadastrados; essa capacidade não equivale ao número de usuários ativos mensais, que determina o volume efetivo de consultas, notificações e consumo. Os cenários financeiros devem, portanto, ser comparados por usuários ativos mensais e assinantes pagantes, sem tratar a capacidade cadastrada como base ativa.

A Tabela 5 projeta os custos operacionais mensais (OpEx) para a faixa inicial de infraestrutura com capacidade de até 50.000 usuários cadastrados. Trata-se de um limite de capacidade contratada, não de uma projeção de 50.000 usuários ativos mensais.

Tabela 5 - Custos operacionais mensais (OpEx) estimados por categoria

Categoria de Custo	Especificação Técnica / Provedor	Custo Mensal Estimado (R\$)	Natureza do Custo
Hospedagem e Banco	Instâncias gerenciadas de PostgreSQL e <i>Cache</i> Redis (Railway/AWS)	R\$ 180,00	Fixo (Escalonável)
API Have I Been Pwned	Assinatura comercial da API de consulta OSINT corporativa	R\$ 25,00 (\$ 4,00 USD)	Fixo
API OpenAI — GPT-4o-mini (IA)	Consumo de tokens do modelo GPT-4o-mini, via SDK da OpenAI 6.41.0 (média de consultas)	R\$ 150,00	Variável (Por Demanda)
Serviços de Alerta	Disparos de SMS e Notificações via WhatsApp (Infobip/Twilio)	R\$ 200,00	Variável (Por Demanda)
Marketing e Tráfego	Campanhas de aquisição orgânica e tráfego pago (insumo do CAC, Seção 5.6)	R\$ 400,00	Flexível

Manutenção e Suporte	Licenças de software de suporte e provisionamento de CI/CD	R\$ 100,00	Fixo
TOTAL MENSAL	Custo Operacional de Sustentação Inicial	R\$ 1.055,00	—

Memória de cálculo auditável: custo de IA = número de consultas à IA × custo médio por consulta; com as premissas existentes, $60.000 \times R\$ 0,0025 = R\$ 150,00/\text{mês}$. Custo de mensagens = número de notificações × custo unitário; com as premissas existentes, $1.000 \times R\$ 0,20 = R\$ 200,00/\text{mês}$. HIBP = número de chaves ou planos necessários para a faixa de volume × preço unitário do plano; o orçamento atual considera uma assinatura de R\$ 25,00, devendo a quantidade ser revista por faixa de volume. Infraestrutura/hospedagem = custo-base da faixa de capacidade, revisto quando CPU, memória, armazenamento ou transferência ultrapassarem o limite contratado. Como referência, 60.000 consultas mensais equivalem a 4 consultas por usuário ativo para uma base de 15.000 usuários ativos mensais (Cenário Realista); para outras bases, o custo de IA deve ser recalculado como usuários ativos × consultas por usuário × R\$ 0,0025, e o de mensagens como usuários ativos notificados × R\$ 0,20. São premissas de planejamento, sujeitas ao volume efetivo, ao contrato, ao câmbio e, no caso da IA, à composição de tokens de entrada e saída.

Nota de Otimização Arquitetural: O custo com a API do modelo GPT-4o-mini, acessado pelo SDK oficial da OpenAI, versão 6.41.0 é mitigado diretamente pela camada de *cache* em memória (**Redis**) projetada no back-end. Como buscas repetidas pelo mesmo e-mail não disparam novas chamadas à LLM, o consumo de tokens variáveis é drasticamente reduzido.

5.2 Receitas

A geração de receita do SafeID apoia-se na conversão de usuários da base gratuita para o plano recorrente **SafeID Pro**. O valor da assinatura mensal foi precificado a **R\$ 11,90/mês**, um patamar competitivo e acessível para a realidade socioeconômica do cidadão brasileiro leigo, posicionando o sistema como um serviço de proteção essencial de baixo custo.

A divisão de entregáveis entre as camadas de serviço configura-se da seguinte forma:

- **Plano Gratuito (Social):** Permite ao cidadão realizar 1 varredura manual de e-mail a cada 24 horas. Exibe o indicador visual do *Risk Score* e os metadados técnicos brutos das vulnerabilidades encontradas.
- **Plano Premium (SafeID Pro - Recorrente):** R\$ 11,90 mensais. Inclui monitoramento automatizado e contínuo em tempo real de até 3 contas/documentos na *Dark Web*; envio imediato de alertas via WhatsApp/SMS assim que um novo vazamento global for

detectado; e acesso ilimitado aos manuais de ação didáticos e personalizados gerados pela Inteligência Artificial.

5.3 Cenário Realista

O Cenário Realista distingue capacidade e utilização: a infraestrutura pode suportar até 50.000 usuários cadastrados, mas a projeção financeira usa usuários ativos mensais e assinantes pagantes. A conversão de 2% aplica-se à base ativa mensal considerada em cada marco, e não ao limite de capacidade cadastrada; assim, evita-se comparar indevidamente capacidade técnica com procura efetiva.

No sexto mês, a projeção existente considera 15.000 usuários ativos mensais e 300 assinantes pagantes, pois $15.000 \times 2\% = 300$. A capacidade de até 50.000 cadastrados permanece apenas como referência de infraestrutura.

- No M6, a receita mensal bruta é: $300 \text{ assinantes} \times \text{R\$ } 11,90 = \text{R\$ } 3.570,00$.
- **Marcos auditáveis do cenário realista:** M1 — 30 assinantes, receita de $30 \times \text{R\$ } 11,90 = \text{R\$ } 357,00$, custo de R\$ 1.055,00 e resultado de $-\text{R\$ } 698,00$; M3 — 90 assinantes, receita de $90 \times \text{R\$ } 11,90 = \text{R\$ } 1.071,00$, custo de R\$ 1.055,00 e resultado de R\$ 16,00; M6 — 300 assinantes, receita de $300 \times \text{R\$ } 11,90 = \text{R\$ } 3.570,00$, custo reajustado de R\$ 1.400,00 e resultado de R\$ 2.170,00. Com esses valores, a alegação de equilíbrio no terceiro mês é sustentada por resultado positivo de R\$ 16,00.

A trajetória preservada é: M1 = 30 assinantes, M2 = 60, M3 = 90, M4 = 160, M5 = 230 e M6 = 300. À taxa de conversão de 2%, os marcos M1, M3 e M6 correspondem, respectivamente, a 1.500, 4.500 e 15.000 usuários ativos mensais. O crescimento pressupõe aquisição progressiva e retenção, mas não transforma automaticamente usuários cadastrados em usuários ativos. O custo reajustado de R\$ 1.400,00 no M6 é uma premissa de planejamento que acrescenta R\$ 345,00 ao custo inicial de R\$ 1.055,00, refletindo o aumento de consultas à IA, de notificações e a eventual revisão da faixa de hospedagem (Seção 5.1).

5.4 Cenário Otimista

O Cenário Otimista também separa capacidade e utilização: a infraestrutura mantém capacidade para até 50.000 usuários cadastrados, enquanto receitas e custos variáveis devem ser comparados pela base ativa mensal e pelos assinantes pagantes. A taxa de conversão de 4,5% aplica-se à base ativa; os 1.350 assinantes projetados implicam 30.000 usuários ativos mensais, pois $30.000 \times 4,5\% = 1.350$. A projeção pressupõe forte tração e expansão B2B2C, sem equiparar o limite de cadastrados à base ativa.

Ao final do primeiro ano, o cenário preserva a capacidade de até 50.000 usuários cadastrados e projeta 30.000 usuários ativos mensais, dos quais 1.350 seriam assinantes pagantes.

- **Faturamento Mensal Bruto:** $1.350 \text{ assinantes} \times \text{R\$ } 11,90 = \text{R\$ } 16.065,00$.
- Com 1.350 assinantes, a receita mensal é R\$ 16.065,00. Mantida a estimativa existente de R\$ 2.800,00 para o custo operacional com o aumento de IA e mensagens, o resultado mensal é $\text{R\$ } 16.065,00 - \text{R\$ } 2.800,00 = \text{R\$ } 13.265,00$. O ganho de escala não elimina a revisão da faixa de hospedagem quando CPU, memória, armazenamento ou transferência excederem os limites contratados.

O superávit financeiro deste cenário viabiliza o reinvestimento imediato em Pesquisa e Desenvolvimento (P&D), direcionado às evoluções descritas na Seção 6.5 (Trabalhos Futuros).

5.5 Cenário Pessimista

O Cenário Pessimista usa 10.000 usuários ativos mensais, e não a capacidade de até 50.000 cadastrados. Com conversão de 0,6%, obtêm-se $10.000 \times 0,006 = 60$ assinantes pagantes. Ao preço de R\$ 11,90, a receita é $60 \times \text{R\$ } 11,90 = \text{R\$ } 714,00$; diante do custo de R\$ 1.055,00, o resultado é $\text{R\$ } 714,00 - \text{R\$ } 1.055,00 = -\text{R\$ } 341,00$.

O custo mensal de R\$ 1.055,00 representa a faixa inicial e componentes variáveis; não é integralmente fixo. A hospedagem parte do custo-base da faixa de capacidade e deve ser revista quando CPU, memória, armazenamento ou transferência ultrapassarem o limite contratado. IA e mensagens variam com consultas e notificações, e o HIBP varia com o número de chaves ou planos exigidos pela faixa de volume.

Plano de Mitigação de Riscos (Ações Contingenciais)

Para reverter o déficit do cenário pessimista e assegurar a sobrevivência do produto sem comprometer a qualidade de entrega, a equipe gestora do SafeID estabelece as seguintes ações estruturadas:

1. **Redução de Infraestrutura (*Downscaling*):** Migração temporária dos servidores lógicos para planos mínimos de computação em nuvem e limitação agressiva da retenção de dados temporários de histórico no PostgreSQL, reduzindo o custo fixo de hospedagem pela metade.
2. **Otimização de Custos de IA:** Redução do limite de tokens de saída (*Max Tokens*) do modelo e aumento da agressividade das políticas de *cache* do Redis, estabelecendo como meta que a inteligência artificial só seja acionada em casos críticos e estritamente inéditos.

5.6 Métricas SaaS: CAC e LTV

CAC (Custo de Aquisição de Cliente) = verba de marketing atribuível ao período $\div$ novos clientes pagantes adquiridos no mesmo período. Com a verba mensal de R\$ 400,00 prevista na Tabela 5 e a trajetória de assinantes do Cenário Realista (Seção 5.3), obtêm-se um CAC projetado de R\$ 13,33 entre M1 e M3 ($\text{R\$ } 400,00 \div 30$ novos assinantes por mês) e de R\$ 5,71 entre M4 e M6 ($\text{R\$ } 400,00 \div 70$ novos assinantes por mês). Trata-se de estimativa de projeção, que atribui toda a verba de marketing às novas assinaturas e desconsidera cancelamentos; o valor real deverá ser apurado por canal quando houver dados de aquisição observados.

LTV (Lifetime Value) = receita média mensal por assinante $\times$ margem bruta $\times$ tempo médio de permanência, com permanência $\approx 1 \div$ churn mensal. Como ainda não existem dados observados de churn e de margem, adota-se a hipótese ilustrativa de churn mensal de 5% (permanência média de 20 meses) e margem bruta de 70%, o que resulta em $\text{LTV} \approx \text{R\$ } 11,90 \times 0,70 \times 20 = \text{R\$ } 166,60$ por assinante. Essas hipóteses são de planejamento e devem ser substituídas por valores observados após o lançamento.

Como o SafeID está em fase pré-operacional, ainda não há dados históricos suficientes de churn, retenção, margem realizada e aquisição incremental por canal. Portanto, os valores de CAC e LTV apresentados são projeções baseadas nas premissas do Cenário Realista e em hipóteses explícitas, e não resultados observados; a razão LTV/CAC somente deverá ser divulgada depois da validação com dados reais.

6 CONSIDERAÇÕES FINAIS

Este capítulo sintetiza os resultados alcançados com o desenvolvimento do SafeID e avalia sua correspondência com os objetivos estabelecidos para o projeto. Nas seções seguintes, são examinados o atendimento aos objetivos específicos, os resultados e limitações observados, as escolhas e os descartes realizados, as contribuições da solução e as possibilidades de continuidade do trabalho.

6.1 Verificação dos Objetivos Específicos

Concluído na Fase PI-1 — Investigar, selecionar e consumir APIs OSINT.

Concluído na Fase PI-1 — Modelar e implementar o algoritmo de Risk Score.

Concluído na Fase PI-1 — Desenvolver a interface web responsiva e a visualização do risco.

Concluído na Fase PI-1 — Projetar a arquitetura de persistência e implementar a base para histórico, sem pressupor a conclusão integral das funções de conta e histórico previstas para a PI-2.

Planejado para PI-2 — Integrar a LLM como assistente cognitivo.

Planejado para PI-2 — Aplicar integralmente *Privacy by Design* e as diretrizes da LGPD ao ciclo operacional.

Nenhum objetivo específico foi classificado como “Não Iniciado”, pois os itens pendentes já possuem planejamento formal para a PI-2.

A conclusão deste projeto de engenharia de software consolida a entrega correspondente à etapa PI-1 e o amadurecimento técnico e analítico da equipe de desenvolvimento. O projeto cumpriu com êxito os objetivos da etapa PI-1, enquanto os demais estão estrategicamente planejados para a PI-2.

6.2 Dificuldades

O processo de desenvolvimento de um ecossistema pautado em *Security by Design* exigiu constantes avaliações de impacto entre as dimensões de custos, performance, segurança e prazos. Diante disso, o grupo deparou-se com desafios complexos que exigiram escolhas arquiteturais rígidas e o descarte consciente de soluções inicialmente planejadas.

6.2.1 Dificuldades Técnicas Encontradas

As principais dificuldades técnicas enfrentadas durante o desenvolvimento são descritas a seguir:

- **Gerenciamento de Taxa de Limite (*Rate Limiting*) das APIs Externas:** A principal barreira operacional residiu na integração com a API *Have I Been Pwned*. Por tratar-se de um serviço comercial com severas restrições de requisições por segundo sob pena de bloqueio de chave, tornou-se inviável adotar uma arquitetura de chamadas síncronas

diretas a partir do front-end. A solução desse impasse exigiu uma reestruturação profunda do *back-end*, introduzindo uma camada de mensageria assíncrona baseada em filas (**BullMQ** e **Redis**) para cadenciar e suavizar o fluxo de consultas em conformidade com as diretrizes do provedor OSINT.

- **Conformidade com a LGPD:** O alinhamento integral do sistema às diretrizes da LGPD está planejado para a PI-2. Os controles técnicos descritos apoiam a conformidade, mas não constituem, isoladamente, comprovação jurídica integral, que também depende de governança, processos, bases legais e avaliação especializada.
- **Determinismo e Custos em Inteligência Artificial:** O controle da imprevisibilidade das respostas geradas por Grandes Modelos de Linguagem (LLMs) será objeto de validação na PI-2. Nessa etapa, deverão ser realizadas rodadas de parametrização de temperatura (temperatura $\approx 0,2$) e definida uma estrutura de saída rígida (Structured Outputs), visando a um comportamento determinista, didático e financeiramente sustentável para o projeto.

6.3 Escolhas e Descartes

Esta seção sistematiza as principais decisões tomadas durante a concepção e a implementação do SafeID, considerando sua aderência aos requisitos, às restrições do projeto e aos objetivos acadêmicos definidos. As subseções seguintes apresentam, de forma complementar, as escolhas arquiteturais consolidadas e as alternativas descartadas, acompanhadas das justificativas que fundamentaram essas decisões.

6.3.1 Escolhas Arquiteturais Consolidadas

As escolhas arquiteturais consolidadas durante o desenvolvimento são apresentadas a seguir, com as respectivas justificativas:

- **Adoção da *Clean Architecture*:** A escolha por isolar as regras de negócio (*Core Domain*) das ferramentas de infraestrutura (bancos de dados e APIs de IA) mostrou-se o acerto mais determinante do projeto. Essa dissociação permitiu que os testes unitários e de mutação (*Fuzz Testing*) fossem executados de forma substancialmente isolada e veloz através do *pipeline* do **GitHub Actions**, blindando o sistema contra regressões sem a necessidade de acionar serviços externos pagos a cada build.
- **Uso Estratégico do Modelo GPT-4o-mini (SDK oficial da OpenAI, versão 6.41.0):** A adoção desse modelo permanece prevista para a PI-2, em razão do equilíbrio esperado entre custo computacional, velocidade de inferência e capacidade de síntese textual. Sua aplicação deverá limitar-se à contextualização didática dos metadados previamente higienizados pelo *back-end*, e sua viabilidade será objeto de validação nessa etapa.

6.3.2 Descartes Estratégicos

Em contrapartida, as alternativas descartadas e as razões do descarte são descritas a seguir:

- **Implementação Gradual da Infraestrutura AWS:** O planejamento arquitetural do projeto estabeleceu o ecossistema da Amazon Web Services (AWS) como a infraestrutura alvo (Target Architecture) sujeita a confirmação para o ambiente de produção do SafeID, conforme detalhado na modelagem topológica do sistema (ECS Fargate, VPC isolada e RDS PostgreSQL). No entanto, para a fase de MVP (Minimum Viable Product) e validação do ecossistema, a equipe adotou uma estratégia de implantação incremental e financeiramente sustentável. Em vez de provisionar imediatamente os recursos corporativos de alta disponibilidade da AWS — o que geraria custos mínimos elevados em dólar e complexidade precoce de gerenciamento de redes virtuais para um ambiente de testes —, optou-se pelo empacotamento agnóstico via contêineres Docker Multi-stage. Essa abordagem permitiu hospedar a aplicação temporariamente em plataformas PaaS (como Railway e Vercel), contribuindo para agilidade no ciclo de desenvolvimento, custo reduzido e total isolamento tecnológico.
- **Descarte da Fragmentação Prematura em Microsserviços:** Considerou-se inicialmente construir o SafeID em uma arquitetura puramente distribuída baseada em microsserviços (onde módulos de Inteligência Artificial, autenticação de usuários e serviços de notificação rodariam em servidores físicos isolados). Essa abordagem foi prontamente descartada porque introduziria uma complexidade de orquestração (como o uso obrigatório de Kubernetes ou múltiplos *pipelines* de CI/CD) desproporcional à necessidade atual de tráfego do sistema. O descarte dessa distribuição excessiva abriu espaço para a adoção de um **Monólito Modular** altamente coeso. Sob carga extrema, o processamento das integrações externas não afeta a renderização do sistema porque foi adotada uma solução híbrida: o servidor permanece monolítico e de fácil implantação, mas o trabalho pesado é distribuído logicamente em filas assíncronas geridas por Redis e BullMQ. Essa escolha favoreceu o desempenho e reduziu os custos operacionais do projeto.

6.4 Limitações

Nesta etapa, o SafeID apresenta quatro limitações principais: dependência atual de uma única API de *threat intelligence*, a Have I Been Pwned (HIBP); cobertura de branches de 81,90% na suíte unitária, com 40 caminhos de infraestrutura e de integrações externas ainda não exercitados, valor obtido com os *endpoints* e e-mails de teste da API HIBP, que é paga, e

portanto tratado como estimativa preliminar; ausência, até o momento, de testes com usuários reais e de validação empírica de usabilidade; e escopo restrito ao e-mail como identificador primário.

6.5 Trabalhos Futuros

Como o código e o banco de dados foram estruturados de forma portátil, a transição para a infraestrutura nativa da AWS ocorrerá de forma gradual, acompanhando a evolução do projeto rumo ao ambiente produtivo em escala real.

Como continuidade imediata, a PI-2 prevê a integração da LLM e a geração do plano de ação (itens 14 e 15 do *Backlog*), a autenticação e o histórico criptografado, a gestão de múltiplos identificadores por usuário (US07), os alertas ativos por e-mail e os testes de carga e de penetração em ambiente de implantação. Em evolução posterior, recomendam-se a validação de usabilidade com usuários reais, a incorporação de uma segunda fonte de *threat intelligence* para reduzir a dependência da HIBP e a ampliação dos identificadores aceitos além do e-mail. No campo da qualidade, prevê-se cobrir os 40 branches restantes e elevar o limite de branches do Jest de 68% para 80%, progredindo gradualmente para cerca de 90% de cobertura. Também estão previstos o monitoramento de chaves PIX e o desenvolvimento de funcionalidades B2B2C.

REFERÊNCIAS BIBLIOGRÁFICAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR ISO/IEC 25010: engenharia de sistemas e software — requisitos e avaliação da qualidade de produto de software (SQuaRE)**. Rio de Janeiro: ABNT, 2011. Disponível em: <https://www.abntcatalogo.com.br/>. Acesso em: 15 set. 2026.

AUTORIDADE NACIONAL DE PROTEÇÃO DE DADOS. **Guia orientativo para definições dos agentes de tratamento de dados pessoais e do encarregado**. Versão 2.0. Brasília, DF: ANPD, 2022. Disponível em: <https://www.gov.br/anpd/pt-br/centrais-de-conteudo/materiais-educativos-e-publicacoes/guia-orientativo-para-definicoes-dos-agentes-de-tratamento-de-dados-pessoais-e-do-encarregado>. Acesso em: 15 set. 2026.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Dispõe sobre a proteção de dados pessoais e altera a Lei nº 12.965, de 23 de abril de 2014 (Marco Civil da Internet). Brasília, DF: Presidência da República, 2018. Disponível em: http://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/113709.htm. Acesso em: 15 set. 2026.

CHEN, Jing; HENRY, Elaine; JIANG, Xi. Is Cybersecurity Risk Factor Disclosure Informative? Evidence from Disclosures Following a Data Breach. **Journal of Business Ethics**, v. 187, n. 1, p. 199–224, 2023.

CHEN, Qiang; WANG, Lun. Social response and management of cybersecurity incidents. **Academic Journal of Sociology and Management**, v. 2, n. 4, p. 49–56, 2024.

CONTI, Mauro; DARGAHI, Tooska; DEGHANTANHA, Ali. Cyber threat intelligence: challenges and opportunities. **Cyber Threat Intelligence**, [s. l.], p. 1–6, 2018a.

CONTI, Mauro et al. Shadow Attacks Based on Password Reuses: A Quantitative Empirical Analysis. **IEEE Transactions on Dependable and Secure Computing**, v. 15, n. 2, p. 309–320, 2018b.

FIRST — FORUM OF INCIDENT RESPONSE AND SECURITY TEAMS. **Common Vulnerability Scoring System v3.1: Specification Document**. [S. l.]: FIRST, 2019. Disponível em: <https://www.first.org/cvss/v3.1/specification-document>. Acesso em: 15 set. 2026.

FORTINET. Brasil é o segundo país que mais sofre ataques cibernéticos na América Latina. São Paulo, 18 ago. 2022. Disponível em: <https://www.fortinet.com/br/corporate/about-us/newsroom/press-releases/2022/brasil-e-o-segundo-pais-que-mais-sofre-ataques-ciberneticos-na-a>. Acesso em: 8 out. 2026.

GRAFANA LABS. **Grafana k6 documentation**. [S. l.]: Grafana Labs, [2026]. Disponível em: <https://grafana.com/docs/k6/latest/>. Acesso em: 15 set. 2026.

HAVE I BEEN PWNED. **API Documentation — Version 3**. [S. l.], [s. d.]. Disponível em: <https://haveibeenpwned.com/API/v3>. Acesso em: 05 mai. 2026.

IBM SECURITY. **Cost of a Data Breach Report 2023**. 2023. Disponível em: <https://www.ibm.com/security/data-breach>. Acesso em: 18 abr. 2026.

JEST. **Getting started**. [S. l.]: Jest, 2026. Disponível em: <https://jestjs.io/docs/getting-started>. Acesso em: 15 set. 2026.

META. **React: A JavaScript library for building user interfaces**. [S. l.], [s. d.]. Disponível em: <https://react.dev/>. Acesso em: 05 mai. 2026.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY (NIST). **NIST Special Publication 800-63B: Digital Identity Guidelines — Authentication and Lifecycle Management**. Gaithersburg: NIST, 2017. Disponível em: <https://pages.nist.gov/800-63-3/sp800-63b.html>. Acesso em: 15 set. 2026.

NEMEC ZLATOLAS, Lili; WELZER, Tatjana; LHOTSKA, Lenka. Data breaches in healthcare: security mechanisms for attack mitigation. **Cluster Computing**, v. 27, n. 7, p. 8639-8654, 2024.

NODE.JS. **Node.js® is a JavaScript runtime built on Chrome's V8 JavaScript engine**. [S. l.], [s. d.]. Disponível em: <https://nodejs.org/>. Acesso em: 05 mai. 2026.

NODESHIFT. **Opossum: Node.js circuit breaker**. [S. l.]: NodeShift, 2026. Disponível em: <https://github.com/nodeshift/opossum/blob/main/README.md>. Acesso em: 15 set. 2026.

PLAYWRIGHT. **Installation: Playwright Test**. [S. l.]: Microsoft, 2026. Disponível em: <https://playwright.dev/docs/intro>. Acesso em: 15 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL: The World's Most Advanced Open Source Relational Database**. [S. l.], [s. d.]. Disponível em: <https://www.postgresql.org/>. Acesso em: 05 mai. 2026.

PRISMA. **Prisma: Next-generation ORM for Node.js & TypeScript**. [S. l.], [s. d.]. Disponível em: <https://www.prisma.io/>. Acesso em: 05 mai. 2026.

REUBEN-OWOH, Blessing; HAIG, Ella. A Systematic Review of Voluntary Cybersecurity Standards and Frameworks. **International Journal of Information Security**, v. 24, n. 5, p. 206, 2025.

SANTOS, J. F.; PINTO, G. Privacidade e segurança em sistemas de informação: uma análise dos aspectos psicológicos e cognitivos envolvidos na percepção e na adoção de medidas de segurança. **Revista Interface Tecnológica**, v. 21, n. 1, p. 288–299, 2025.

TAILWIND LABS. **Tailwind CSS: Rapidly build modern websites without ever leaving your HTML**. [S. l.], [s. d.]. Disponível em: <https://tailwindcss.com/>. Acesso em: 05 mai. 2026.

VON SOLMS, Rossouw; VAN NIEKERK, Johan. From information security to cyber security. **Computers & Security**, v. 38, p. 97–102, 2013.